A Project Report On

Machine Learning Techniques for Cyber Attacks Detection

Submitted to Osmania University for the partial fulfillment

Of the requirement for the Award of Degree for

Master of Science

(Computer Science)

Submitted By

SARIKONDA ABHINAY KUMAR RAJU

(110521504032)

Under the Nobel Guidance of

Mr. S. Rambhopal Reddy

JAGRUTI DEGREE & PG COLLEGE

(Affiliated to Osmania University)

Narayanaguda, Hyderabad-500029

2021-2023

JAGRUTI DEGREE & PG COLLEGE

(Affiliated to Osmania University)

CERTIFICATE

This is to certify that the dissertation entitled

Machine Learning Techniques for Cyber Attacks Detection

was successfully carried out by

SARIKONDA ABHINAY KUMAR RAJU

(110521504032)

At

JAGRUTI DEGREE & PG COLLEGE

In partial fulfillment for the award of degree of Master of Science in

Computer Science for the academic year 2021 – 23

Internal Guide:

Head of the Department **Principal**

Internal Examiner: **External Examiner:**

(company certificate)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose constant guidance and encouragement crown all the efforts with success.

The acknowledgement transcends the reality of formality when we would like to express the deep gratitude and respect to all those people behind the screen who guided, inspired wand helped me for the completion of my project work.

It gives me immense pleasure to express my deep sense of gratitude and indebtedness to the Management, the Principal and Director, for his extensive support and encouragement throughout my course at Jagruti Degree & PG College.

I would like to express my sincere thanks to Mrs. B. Haritha Yadav, Mrs. P.V. Aparanjini Priyadarshini, Mr. S. Rambhopal Reddy, Faculty members of Dept. of Computer Science, Jagruti Degree & PG College, for their suggestions, constant encouragement and immense help throughout the course of project.

Last but not the least, I would like to express any sincere thanks to our college staff and all those contributed in completing this project either directly or indirectly.

DECLARATION

I **Sarikonda Abhinay Kumar Raju** bearing the Hall Ticket No. (**110521504032**), studying in **Jagruti Degree & PG college (JDPC),** declare that the report of the project entitled **"Machine Learning Techniques for Cyber Attacks Detection**" is the result of the original works done by me and has not been submitted earlier to Osmania University or any other Institution for the award of any degree of diploma and does not from part of any other course. This project is submitted in partial fulfillment for the award of degree of Master of Science (Computer Science) from Osmania University, Hyderabad.

(SARIKONDA ABHINAY KUMAR RAJU)

Machine Learning Techniques for Cyber Attacks Detection

ABSTRACT

Contrasted with the past, improvements in PC and correspondence innovations have given broad and propelled changes. The use of new innovations gives incredible advantages to people, organizations, and governments, be that as it may, messes some up against them. For instance, the protection of significant data, security of put away information stages, accessibility of information and so forth. Contingent upon these issues, digital fear based oppression is one of the most significant issues in this day and age. Digital fear, which made a great deal of issues people and establishments, has arrived at a level that could undermine open and nation security by different gatherings, for example, criminal association, proficient people and digital activists. Along these lines, Intrusion Detection Systems (IDS) has been created to maintain a strategic distance from digital assaults. Right now, learning the bolster support vector machine (SVM) calculations were utilized to recognize port sweep endeavors dependent on the new CICIDS2017 dataset with 97.80%, 69.79% precision rates were accomplished individually. Rather than SVM we can introduce some other algorithms like random forest, CNN, ANN where these algorithms can acquire accuracies like SVM – 93.29, CNN – 63.52, Random Forest – 99.93, ANN – 99.11.

CONTENTS

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Introduction about Project

Contrasted with the past, improvements in PC and correspondence innovations have given broad and propelled changes. The use of new innovations gives incredible advantages to people, organizations, and governments, be that as it may, messes some up against them. For instance, the protection of significant data, security of put away information stages, accessibility of information and so forth. Contingent upon these issues, digital fear based oppression is one of the most significant issues in this day and age. Digital fear, which made a great deal of issues people and establishments, has arrived at a level that could undermine open and nation security by different gatherings, for example, criminal association, proficient people and digital activists. Along these lines, Intrusion Detection Systems (IDS) has been created to maintain a strategic distance from digital assaults. Right now, learning the bolster support vector machine (SVM) calculations were utilized to recognize port sweep endeavors dependent on the new CICIDS2017 dataset with 97.80%, 69.79% precision rates were accomplished individually. Rather than SVM we can introduce some other algorithms like random forest, CNN, ANN where these algorithms can acquire accuracies like SVM – 93.29, CNN – 63.52, Random Forest – 99.93, ANN – 99.11.

1.1.1 MOTIVATION

The use of new innovations gives incredible advantages to people, organizations, and governments, be that as it may, messes some up against them. For instance, the protection of significant data, security of put away information stages, accessibility of information and so forth. Contingent upon these issues, digital fear based oppression is one of the most significant issues in this day and age. Digital fear, which made a great deal of issues people and establishments, has arrived at a level that could undermine open and nation security by different gatherings, for example, criminal association, proficient people and digital activists. Along these lines, Intrusion Detection Systems (IDS) has been created to maintain a strategic distance from digital assaults.

1.2 Project Scope

Objective of this project is to detect cyber attacks by using machine learning algorithms like

- ANN
- CNN
- Random forest

ANN: Artificial Neural Network (ANN) is a machine learning model that mimics the structure and function of biological neural networks. It consists of interconnected layers of artificial neurons that can process data and learn from it. ANN can be used for cyber attacks detection by learning the patterns of normal and malicious network traffic and classifying them accordingly. ANN can also handle nonlinear and complex data, which is common in network security.

CNN: Convolutional Neural Network (CNN) is a type of deep learning model that uses convolutional layers to extract features from data, such as images, text, or network packets. CNN can be used for cyber attacks detection by applying filters to the raw data and capturing the spatial and temporal dependencies. CNN can also reduce the dimensionality and noise of the data, which can improve the accuracy and efficiency of the detection.

Random forest: Random Forest is an ensemble learning method that combines multiple decision trees to create a more robust and accurate classifier. Random forest can be used for cyber attacks detection by splitting the data into subsets and building a tree for each subset. The trees are then aggregated by voting or averaging to produce the final prediction. Random forest can also handle imbalanced and noisy data, as well as feature selection.

1.3 Module Description

The actors identified in this system are:

- System Administrator
- Customer
- Customer Care

System Administrator: A system administrator is an individual responsible for managing and maintaining the operation, configuration, and security of a computer system or network. The system administrator is likely responsible for overseeing the implementation, monitoring, and maintenance of the intrusion detection system (IDS) used for cyber attack detection. They may handle tasks such as configuring the system, managing data collection and preprocessing, training and testing machine learning models, and ensuring the system's overall performance and security.

Customer: Customer refers to an entity or organization that utilizes or benefits from the developed machine learning-based cyber attack detection system. The customer may be an enterprise, government agency, or any other entity that requires robust cybersecurity measures to protect their systems and data. The customer may interact with the system to access reports or alerts regarding detected cyber attacks, monitor the system's performance, and provide feedback for system improvements or enhancements.

Customer Care: customer care may involve a dedicated team or department responsible for providing technical support, resolving issues related to the machine learning-based cyber attack detection system, and assisting customers in effectively utilizing and managing the system. Customer care may include tasks such as troubleshooting, responding to customer inquiries, providing guidance on system configuration or usage, and coordinating with the system administrator for resolving more complex technical issues.

CHAPTER 2
Literature Survey/Related Work

2.1 R. Christopher, "Port scanning techniques and the defense against them," SANS Institute, 2001.

Port Scanning is one of the most popular techniques attackers use to discover services that they can exploit to break into systems. All systems that are connected to a LAN or the Internet via a modem run services that listen to well-known and not so well-known ports. By port scanning, the attacker can find the following information about the targeted systems: what services are running, what users own those services, whether anonymous logins are supported, and whether certain network services require authentication. Port scanning is accomplished by sending a message to each port, one at a time. The kind of response received indicates whether the port is used and can be probed for further weaknesses. Port scanners are important to network security technicians because they can reveal possible security vulnerabilities on the targeted system. Just as port scans can be ran against your systems, port scans can be detected and the amount of information about open services can be limited utilizing the proper tools. Every publicly available system has ports that are open and available for use. The object is to limit the exposure of open ports to authorized users and to deny access to the closed ports.

2.2 S. Staniford, J. A. Hoagland, and J. M. McAlerney, "Practical automated detection of stealthy port scans," Journal of Computer Security, vol. 10, no. 1-2, pp. 105–136, 2002.

Port scanning is a common activity of considerable importance. It is often used by computer attackers to characterize hosts or networks which they are considering hostile activity against. Thus, it is useful for system administrators and other network defenders to detect port scans as possible preliminaries to a more serious attack. It is also widely used by network defenders to understand and find vulnerabilities in their own networks. Thus, it is of considerable interest to attackers to determine whether or not the defenders of a network are port scanning it regularly. However, defenders will not usually wish to hide their port scanning, while attackers will. For definiteness, in the remainder of this paper, we will speak of the attackers scanning the network, and the defenders trying to detect the scan. There are several legal/ethical debates about portscanning which break out regularly on Internet mailing lists and newsgroups. One concerns whether port scanning of remote networks without permission from the owners is itself a legal and ethical activity. This is presently a grey area in most jurisdictions. However, our experience from following up on unsolicited remote port scans we detect in practice is that almost all of them turn out to have come from compromised hosts and thus are very likely to be hostile. So we think it reasonable to consider a port scan as at least potentially hostile, and to report it to the administrators of the remote network from whence it came. However, this paper is focused on the technical questions of how to detect port scans, which are independent of what significance one imbues them with, or how one chooses to respond to them. Also, we are focused here on the problem of detecting a port scan via a network intrusion detection system (NIDS). We try to take into account some of the more obvious ways an attacker could use to avoid detection, but to remain with an approach that is practical to employ on busy networks. In the remainder of this section, we first define port scanning, give a variety of examples at some length, and discuss ways attackers can try to be stealthy. In the next section, we discuss a variety of prior work on port scan detection. Then we present the algorithms that we propose to use, and give some very preliminary data justifying our approach. Finally, we consider possible extensions to this work, along with other applications that might be considered. Throughout, we assume the reader is familiar with Internet protocols, with basic ideas about network intrusion detection and scanning, and with elementary probability

theory, information theory, and linear algebra. There are two general purposes that an attacker might have in conducting a port scan: a primary one, and a secondary one. The primary purpose is that of gathering information about the reachability and status of certain combinations of IP address and port (either TCP or UDP). (We do not directly discuss ICMP scans in this paper, but the ideas can be extended to that case in an obvious way.) The secondary purpose is to flood intrusion detection systems with alerts, with the intention of distracting the network defenders or preventing them from doing their jobs. In this paper, we will mainly be concerned with detecting information gathering port scans, since detecting flood port scans is easy. However, the possibility of being maliciously flooded with information will be an important consideration in our algorithm design. We will use the term scan footprint for the set of port/IP combinations which the attacker is interested in characterizing. It is helpful to conceptually distinguish the footprint of the scan, from the script of the scan, which refers to the time sequence in which the attacker tries to explore the footprint. The footprint is independent of aspects of the script, such as how fast the scan is, whether it is randomized, etc. The footprint represents the attacker's information gathering requirements for her scan, and she designs a scan script that will meet those requirements, and perhaps other non-information-gathering requirements (such as not being detected by an NIDS). The most common type of port scan footprint at present is a horizontal scan. By this, we mean that an attacker has an exploit for a particular service, and is interested in finding any hosts that expose that service. Thus, she scans the port of interest on all IP addresses in some range of interest. Also at present, this is mainly being done sequentially on TCP port 53 (DNS)

2.3 M. C. Raja and M. M. A. Rabbani, "Combined analysis of support vector machine and principle component analysis for ids," in IEEE International Conference on Communication and Electronics Systems, 2016, pp. 1–5.

Compared to the past security of networked systems has become a critical universal issue that influences individuals, enterprises and governments. The rate of attacks against networked systems has increased melodramatically, and the strategies used by the attackers are continuing to evolve. For example, the privacy of important information, security of stored data platforms, availability of knowledge etc. Depending on these problems, cyber terrorism is one of the most important issues in today's world. Cyber terror, which caused a lot of problems to individuals and institutions, has reached a level that could threaten public and country security by various groups such as criminal organizations, professional persons and cyber activists. Intrusion detection is one of the solutions against these attacks. A free and effective approach for designing Intrusion Detection Systems (IDS) is Machine Learning. In this study, deep learning and support vector machine (SVM) algorithms were used to detect port scan attempts based on the new CICIDS2017 dataset Introduction Network Intrusion Detection System (IDS) is a software-based application or a hardware device that is used to identify malicious behavior in the network [1,2]. Based on the detection technique, intrusion detection is classified into anomaly-based and signature-based. IDS developers employ various techniques for intrusion detection. Information security is the process of protecting information from unauthorized access, usage, disclosure, destruction, modification or damage. The terms "Information security", "computer security" and "information insurance" are often used interchangeably. These areas are related to each other and have common goals to provide availability, confidentiality, and integrity of information. Studies show that the first step of an attack is discovery [1]. Reconnaissance is made in order to get information about the system in this stage. Finding a list of open ports in a system provides very critical information for an attacker. For this reason, there are a lot of tools to identify open ports [2] such as antivirus and IDS. One of these techniques is based on machine learning. Machine learning (ML) techniques can predict and detect threats before they result in major security incidents [3]. Classifying instances into two classes is called binary classification. On the other hand, multi-class classification refers to classifying instances into three or more classes. In this research, we adopt both classifications Information security is the

process of protecting information from unauthorized access, usage, disclosure, destruction, modification or damage. The terms ”Information security”, ”computer security” and ”information insurance” are often used interchangeably. These areas are related to each other and have common goals to provide availability, confidentiality, and integrity of information. Studies show that the first step of an attack is discovery [1]. Reconnaissance is made in order to get information about the system in this stage. Finding a list of open ports in a system provides very critical information for an attacker. For this reason, there are a lot of tools to identify open ports [2] such as antivirus and IDS. II. Litrature Review Sharafaldin et al. [4] used a Random Forest Regressor to determine the best set of features to detect each attack family. The authors examined the performance of these features with different algorithms that included K-Nearest Neighbor (KNN), Adaboost, Multi-Layer Perceptron (MLP), Naïve Bayes, Random Forest (RF), Iterative Dichotomiser 3 (ID3) and Quadratic Discriminant Analysis (QDA). The highest precision value was 0.98 with RF and ID3 [4]. The execution time (time to build the model) was 74.39 s. This is while the execution time for our proposed system using Random Forest is 21.52 s with a comparable processor. Survey on Detecting Port Scan Attempts with Combined Analysis of Support Vector Machine and DOI: 10.9790/0661-2103044246 www.iosrjournals.org 43 | Page Furthermore, our proposed intrusion detection system targets a combined detection process of all the attack families. D. Aksu, S. U¨ stebay, M. A. Aydin, and T. Atmaca[09], There are different but limited studies based on the CICIDS2017 dataset. Some of them were discussed here. D.Aksu et al. showed performances of various machine learning algorithms detecting DDoS attacks based on the CICIDS2017 dataset in their previous work [13].The authors of [13] applied the Multi-Layer Perceptron (MLP) classifier algorithm and a Convolutional Neural Network (CNN) classifier that used the Packet CAPture (PCAP) file of CICIDS2017. The authors selected specified network packet header features for the purpose of their study. Conversely, in our paper, we used the corresponding profiles and the labeled flows for machine and deep learning purposes. According to [13], the results demonstrated that the payload classification algorithm was judged to be inferior to MLP. However, it showed significant ability to distinguish network intrusion from benign traffic with an average true positive rate of 94.5% and an average false positive rate of 4.68%. The auther E. Biglar Beigi, H. Hadian Jazi,Machine [14] learning techniques have the ability to learn the normal and anomalous patterns

automatically by training a dataset to predict an anomaly in network traffic. One important characteristic defining the effectiveness of machine learning techniques is the features extracted from raw data for classification and detection. Features are the important information extracted from raw data. The underlying factor in selecting the best features lies in a trade-off between detection accuracy and false alarm rates. The use of all features on the other hand will lead to a significant overhead and thus reducing the risk of removing important features. Although the importance of feature selection cannot be overlooked, intuitive understanding of the problem is mostly used in the selection of features [16]. The authors in [14] proposed a denial of service intrusion detection system that used the Fisher Score algorithm for features selection and Support Vector Machine (SVM), K-Nearest Neighbor (KNN) and Decision Tree (DT) as the classification algorithm. Their IDS achieved 99.7%, 57.76% and 99% success rates using SVM, KNN and DT, respectively. In contrast, our research proposes an IDS to detect all types of attacks embedded in CICIDS2017, and as shown in the confusion matrix results, achieves 100% accuracy for DDoS attacks using (PCA RF)Mc10 with UDBB.The authors in [15] used a distributed Deep Belief Network (DBN) as the the dimensionality reduction approach. The obtained features were then fed to a multi-layer ensemble SVM. The ensemble SVM was accomplished in an iterative reduce paradigm based on Spark (which is a general distributed in-memory computing framework developed at AMP Lab, UC Berkeley), to serve as a Real Time Cluster Computing Framework that can be used in big data analysis [16]. Their proposed approach achieved an F-measure value equal to 0.921. III. Methods 1.1 CICIDS2017 Dataset The CICIDS2017 dataset is used in our study. The dataset is developed by the Canadian Institute for Cyber Security and includes various common attack types. The CICIDS2017 dataset consists of realistic background traffic that represents the network events produced by the abstract behavior of a total of 25 users. The users' profiles were determined to include specific protocols such as HTTP, HTTPS, FTP, SSH and email protocols. The developers used statistical metrics such as minimum, maximum, mean and standard deviation to encapsulate the network events into a set of certain features which include: 1. The distribution of the packet size 2. The number of packets per flow 3. The size of the payload 4. The request time distribution of the protocols 5. Certain patterns in the payload Moreover, CICIDS2017 covers various attack scenarios that represent

common attack families. The attacks include Brute Force Attack, Heart Bleed Attack, Botnet, DoS Attack, Distributed DoS (DDoS) Attack , Web Attack, and Infiltration Attack. 1.2 SUPPORT VECTOR MACHINE The SVM is already known as the best learning algorithm for binary classification. The SVM, originally a type of pattern classifier based on a statistical learning technique for classification and regression with a variety of kernel functions, has been successfully applied to a number of pattern recognition applications. Recently, it has also been applied to information security for intrusion detection. Support Vector Machine has become one of the popular techniques for anomaly intrusion detection due to their good generalization nature and the ability to overcome the curse of dimensionality .Another positive aspect of SVM is that it is useful for finding a global minimum of the actual risk using structural risk minimization, since it can generalize well with kernel tricks even in high-dimensional spaces under little training sample conditions. The SVM can select Survey on Detecting Port Scan Attempts with Combined Analysis of Support Vector Machine and DOI: 10.9790/0661-2103044246 www.iosrjournals.org 44 | Page appropriate setup parameters because it does not depend on traditional empirical risk such as neural networks [13]. One of the main advantage of using SVM for IDS is its speed, as the capability of detecting intrusions in real-time is very important. SVMs can learn a larger set of patterns and be able to scale better, because the classification complexity does not depend on the dimensionality of the feature space. SVMs also have the ability to update the training patterns dynamically whenever there is a new pattern during classification [14]. 1.2.1 Limitation of Support Vector Machine in IDS SVM is basically supervised machine learning method designed for binary classification. Using SVM in IDS domain has some limitation.SVM being a supervised machine learning method requires labelled information for efficient learning. Pre existing knowledge is required for classification which may not be available all the time[13].SVM has the intrinsic structural limitation of the binary classifier i.e. it can only handle binaryclass classification whereas intrusion detection requires multi-class classification [14].Although there are some improvements, the number of dimensions still affects the performance of SVM-based classifier [16]. SVM treats every feature of data equally. In real intrusion detection datasets, many features are redundant or less important .It would be better if feature weights during SVM training are considered [16]. Training of SVM is timeconsuming for IDS domain and requires large dataset storage.Thus SVM is computationally expensive for

resource-limited ad hoc network[12].Moreover SVM requires the processing of raw features for classification which increases the architecture complexity and decreases the accuracy of detecting intrusion 1.3 DEEP LEARNING Deep learning is an improved machine learning technique for feature extraction, perception and learning of machines. Deep learning algorithms performs their operations using multiple consecutive layers. The layers are interlinked and each layer receives the output of the previous layer as input. It is a great advantage to use efficient algorithms for extracting hierarchical features that best represent data rather than manual features in deep learning methods [7], [8]. There are many application areas for Deep Learning, which covers such as Image Processing, Natural Language Processing, biomedical, Customer Relationship Management automation, Vehicle autonomous systems and others.

2.4 S. Aljawarneh, M. Aldwairi, and M. B. Yassein, "Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model," Journal of Computational Science, vol. 25, pp. 152–160, 2018.

n network security, intrusion detection plays an important role. Feature subsets obtained by different feature selection methods will lead to different accuracy of intrusion detection. Using individual feature selection method can be unstable in different intrusion detection scenarios. In this paper, the idea of ensemble is applied to feature selection to adjust feature subsets. Feature selection is converted into a two-category problem, and odd number of feature selection methods is used for voting method to decide whether a feature is required or discarded. In actual operation, mean decrease impurity, random forest classifier, stability selection, recursive feature elimination and chi-square test are used. Feature subsets obtained from them will be adjusted by our proposed method to get ensemble feature subsets. To test the performance, support vector machine, decision tree, knn and multi-layer perception are used to observe and compare the classification accuracy with ensemble feature subsets. Three intrusion detection data sets, including kddcup99, cidds-001 and unsw_nb15 are used in our experiments. The best result is reflected on cidds-001 with a 99.40% classification accuracy. The investigation shows that our method has a certain improvement on classification accuracy of intrusion detection.In this section, we analyze and evaluate the eleven publicly available IDS datasets since 1998 to demonstrate their shortages and issues that reflect the real need for a comprehensive and reliable dataset. DARPA (Lincoln Laboratory 1998-99): The dataset was constructed for network security analysis and exposed the issues associated with the artificial injection of attacks and benign traffic. This dataset includes email, browsing, FTP, Telnet, IRC, and SNMP activities. It contains attacks such as DoS, Guess password, Buffer overflow, remote FTP, Syn flood, Nmap, and Rootkit. This dataset does not represent real-world network traffic, and contains irregularities such as the absence of false positives. Also, the dataset is outdated for the effective evaluation of IDSs on modern networks, both in terms of attack types and network infrastructure. Moreover, it lacks actual attack data records (McHugh, 2000) (Brown et al., 2009). KDD'99 (University of California, Irvine 1998-99): This dataset is an updated version

of the DARPA98, by processing the tcpdump portion. It contains different attacks such as Neptune-DoS, pod-DoS, SmurfDoS, and buffer-overflow (University of California, 2007). The benign and attack traffic are merged together in a simulated environment. This dataset has a large number of redundant records and is studded by data corruptions that led to skewed testing results (Tavallaee et al., 2009). NSL-KDD was created using KDD (Tavallaee et al., 2009) to address some of the KDD's shortcomings (McHugh, 2000). DEFCON (The Shmoo Group, 2000-2002: The DEFCON-8 dataset created in 2000 contains port scanning and buffer overflow attacks, whereas DEFCON-10 dataset, which was created in 2002, contains port scan and sweeps, bad packets, administrative privilege, and FTP by Telnet protocol attacks. In this dataset, the traffic produced during the "Capture the Flag (CTF)" competition is different from the real world network traffic since it mainly consists of intrusive traffic as opposed to normal background traffic. This dataset is used to evaluate alert correlation techniques (Nehinbe, 2010) (Group, 2000). CAIDA (Center of Applied Internet Data Analysis 2002-2016): This organization has three different datasets, the CAIDA OC48, which includes different types of data observed on an OC48 link in San Jose, the CAIDA DDOS, which includes one-hour DDoS attack traffic split of 5-minute pcap files, and the CAIDA Internet traces 2016, which is passive traffic traces from CAIDA's Equinix-Chicago monitor on the High-speed Internet backbone. Most of CAIDAs datasets are very specific to particular events or attacks and are anonymized with their payload, protocol information, and destination. These are not the effective benchmarking datasets due to a number of shortcomings, see (for Applied Internet Data Analysis (CAIDA), 2002) (for Applied Internet Data Analysis (CAIDA), 2007) (for Applied Internet Data Analysis (CAIDA), 2016) (Proebstel, 2008) (Ali Shiravi and Ghorbani, 2012) for details. LBNL (Lawrence Berkeley National Laboratory and ICSI 2004-2005): The dataset is full header network traffic recorded at a medium-sized site. It does not have payload and suffers from a heavy anonymization to remove any information which could identify an individual IP (Nechaev et al., 2004). CDX (United States Military Academy 2009): This dataset represents the network warfare competitions, that can be utilized to generate modern day labelled dataset. It includes network traffic such as Web, email, DNS lookups, and other required services. The attackers used the attack tools such as Nikto, Nessus, and WebScarab to carry out reconnaissance and attacks automatically. This dataset can be used to test IDS alert rules, but it suffers from the lack of traffic diversity and

volume (Sangster et al., 2009). Kyoto (Kyoto University 2009): This dataset has been created through honypots, so there is no process for manual labelling and anonymization, but it has limited view of the network traffic because only attacks directed at the honeypots can be observed. It has ten extra features such as IDS Detection, Malware Detection, and Ashula Detection than previous available datasets which are useful in NIDS analyToward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization 109 sis and evaluation. The normal traffic here has been simulated repeatedly during the attacks and producing only DNS and mail traffic data, which is not reflected in real world normal network traffic, so there are no false positives, which are important for minimizing the number of alerts (Song et al., 2011) (M. Sato, 2012) (R. Chitrakar, 2012). Twente (University of Twente 2009): This dataset includes three services such as OpenSSH, Apache web server and Proftp using auth/ident on port 113 and captured data from a honeypot network by Netflow. There is some simultaneous network traffic such as auth/ident, ICMP, and IRC traffic, which are not completely benign or malicious. Moreover, this dataset contains some unknown and uncorrelated alerts traffic. It is labelled and is more realistic, but the lack of volume and diversity of attacks is obvious (Sperotto et al., 2009). UMASS (University of Massachusetts 2011): The dataset includes trace files, which are network packets, and some traces on wireless applications (of Massachusetts Amherst, 2011) (Nehinbe, 2011). It has been generated using a single TCP-based download request attack scenario. The dataset is not useful for testing IDS and IPS techniques due to the lack of variety of traffic and attacks (Swagatika Prusty and Liberatore, 2011). ISCX2012 (University of New Brunswick 2012): This dataset has two profiles, the Alpha-profile which carried out various multi-stage attack scenarios, and the Beta-profile, which is the benign traffic generator and generates realistic network traffic with background noise. It includes network traffic for HTTP, SMTP, SSH, IMAP, POP3, and FTP protocols with full packet payload. However, it does not represent new network protocols since nearly 70% of todays network traffics are HTTPS and there are no HTTPS traces in this dataset. Moreover, the distribution of the simulated attacks is not based on real world statistics (Ali Shiravi and Ghorbani, 2012). ADFA (University of New South Wales 2013): This dataset includes normal training and validating data and 10 attacks per vector (Creech and Hu, 2013). It contains FTP and SSH password brute force, Java based Meterpreter, Add new Superuser, Linux Meterpreter payload and C100 Webshel attacks. In addition to the lack of attack

diversity and variety of attacks, the behaviors of some attacks in this dataset are not well separated from the normal behavior (Xie and Hu, 2013) (Xie et al., 2014).

2.5 I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization." in ICISSP, 2018, pp. 108–116.

With exponential growth in the size of computer networks and developed applications, the significant increasing of the potential damage that can be caused by launching attacks is becoming obvious. Meanwhile, Intrusion Detection Systems (IDSs) and Intrusion Prevention Systems (IPSs) are one of the most important defense tools against the sophisticated and ever-growing network attacks. Due to the lack of adequate dataset, anomaly-based approaches in intrusion detection systems are suffering from accurate deployment, analysis and evaluation. There exist a number of such datasets such as DARPA98, KDD99, ISC2012, and ADFA13 that have been used by the researchers to evaluate the performance of their proposed intrusion detection and intrusion prevention approaches. Based on our study over eleven available datasets since 1998, many such datasets are out of date and unreliable to use. Some of these datasets suffer from lack of traffic diversity and volumes, some of them do not cover the variety of attacks, while others anonymized packet information and payload which cannot reflect the current trends, or they lack feature set and metadata. This paper produces a reliable dataset that contains benign and seven common attack network flows, which meets real world criteria and is publicly available. Consequently, the paper evaluates the performance of a comprehensive set of network traffic features and machine learning algorithms to indicate the best set of features for detecting the certain attack categories. 1 INTRODUCTION Intrusion detection plays a vital role in the network defense process by aiming security administrators in forewarning them about malicious behaviors such as intrusions, attacks, and malware. Having IDS is a mandatory line of defense for protecting critical networks against these ever-increasing issues of intrusive activities. So, research on IDS domain has flourished over the years to propose the better IDS systems. However, many researchers struggle to find comprehensive and valid datasets to test and evaluate their proposed techniques (Koch et al., 2017) and having a suitable dataset is a significant challenge itself (Nehinbe, 2011). On one hand, many such datasets cannot be shared due to the privacy issues. On the other hand, those that become available are heavily

anonymized and do not reflect the current trends, even though, the lack of traffic variety and attack diversity is evident in most of them. Therefore, based on the lack of certain statistical characteristics and the unavailability of these The first two authors contributed equally to this work. datasets a perfect dataset is yet to be realized (Nehinbe, 2011; Ali Shiravi and Ghorbani, 2012). It is also necessary to mention that due to malware evolution and the continuous changes in attack strategies, benchmark datasets need to be updated periodically (Nehinbe, 2011). Since 1999, Scott et al. (Scott and Wilkins, 1999), Heideman and Papadopulus (Heidemann and Papdopoulos, 2009), Ghorbani et al. (Ghorbani Ali and Mahbod, 2010), Nehinbe (Nehinbe, 2011), Shiravi et al. (Ali Shiravi and Ghorbani, 2012), and Sharfaldin et al. (Gharib et al., 2016) tried to propose an evaluation framework for IDS datasets. According to the last research and proposed evaluation framework, eleven characteristics, namely Attack Diversity, Anonymity, Available Protocols, Complete Capture, Complete Interaction, Complete Network Configuration, Complete Traffic, Feature Set, Heterogeneity, Labelling, and Metadata are critical for a comprehensive and valid IDS dataset (Gharib et al., 2016). Our Contributions: Our contributions in this paper are twofold. Firstly, we generate a new IDS dataset namely CICIDS2017, which covers all the eleven 108 Sharafaldin, I., Lashkari, A. and Ghorbani, A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. DOI: 10.5220/0006639801080116 In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018), pages 108-116 ISBN: 978-989-758-282-0 Copyright © 2018 by SCITEPRESS – Science and Technology Publications, Lda. All rights reserved necessary criteria with common updated attacks such as DoS, DDoS, Brute Force, XSS, SQL Injection, Infiltration, Port scan and Botnet. The dataset is completely labelled and more than 80 network traffic features extracted and calculated for all benign and intrusive flows by using CICFlowMeter software which is publicly available in Canadian Institute for Cybersecurity website (Habibi Lashkari et al., 2017). Secondly, the paper analyzes the generated dataset to select the best feature sets to detect different attacks and also we executed seven common machine learning algorithms to evaluate our dataset

In this section, we analyze and evaluate the eleven publicly available IDS datasets since 1998 to demonstrate their shortages and issues that reflect the real need for a comprehensive and reliable dataset. DARPA (Lincoln Laboratory 1998-99): The dataset was constructed for network security analysis and exposed the issues

associated with the artificial injection of attacks and benign traffic. This dataset includes email, browsing, FTP, Telnet, IRC, and SNMP activities. It contains attacks such as DoS, Guess password, Buffer overflow, remote FTP, Syn flood, Nmap, and Rootkit. This dataset does not represent real-world network traffic, and contains irregularities such as the absence of false positives. Also, the dataset is outdated for the effective evaluation of IDSs on modern networks, both in terms of attack types and network infrastructure. Moreover, it lacks actual attack data records (McHugh, 2000) (Brown et al., 2009). KDD'99 (University of California, Irvine 1998-99): This dataset is an updated version of the DARPA98, by processing the tcpdump portion. It contains different attacks such as Neptune-DoS, pod-DoS, SmurfDoS, and buffer-overflow (University of California, 2007). The benign and attack traffic are merged together in a simulated environment. This dataset has a large number of redundant records and is studded by data corruptions that led to skewed testing results (Tavallaee et al., 2009). NSL-KDD was created using KDD (Tavallaee et al., 2009) to address some of the KDD's shortcomings (McHugh, 2000). DEFCON (The Shmoo Group, 2000-2002: The DEFCON-8 dataset created in 2000 contains port scanning and buffer overflow attacks, whereas DEFCON-10 dataset, which was created in 2002, contains port scan and sweeps, bad packets, administrative privilege, and FTP by Telnet protocol attacks. In this dataset, the traffic produced during the "Capture the Flag (CTF)" competition is different from the real world network traffic since it mainly consists of intrusive traffic as opposed to normal background traffic. This dataset is used to evaluate alert correlation techniques (Nehinbe, 2010) (Group, 2000). CAIDA (Center of Applied Internet Data Analysis 2002-2016): This organization has three different datasets, the CAIDA OC48, which includes different types of data observed on an OC48 link in San Jose, the CAIDA DDOS, which includes one-hour DDoS attack traffic split of 5-minute pcap files, and the CAIDA Internet traces 2016, which is passive traffic traces from CAIDA's Equinix-Chicago monitor on the High-speed Internet backbone. Most of CAIDAs datasets are very specific to particular events or attacks and are anonymized with their payload, protocol information, and destination. These are not the effective benchmarking datasets due to a number of shortcomings, see (for Applied Internet Data Analysis (CAIDA), 2002) (for Applied Internet Data Analysis (CAIDA), 2007) (for Applied Internet Data Analysis (CAIDA), 2016) (Proebstel, 2008) (Ali Shiravi and Ghorbani, 2012) for details. LBNL (Lawrence Berkeley National Laboratory and ICSI 2004-2005): The dataset is full

header network traffic recorded at a medium-sized site. It does not have payload and suffers from a heavy anonymization to remove any information which could identify an individual IP (Nechaev et al., 2004). CDX (United States Military Academy 2009): This dataset represents the network warfare competitions, that can be utilized to generate modern day labelled dataset. It includes network traffic such as Web, email, DNS lookups, and other required services. The attackers used the attack tools such as Nikto, Nessus, and WebScarab to carry out reconnaissance and attacks automatically. This dataset can be used to test IDS alert rules, but it suffers from the lack of traffic diversity and volume (Sangster et al., 2009). Kyoto (Kyoto University 2009): This dataset has been created through honypots, so there is no process for manual labelling and anonymization, but it has limited view of the network traffic because only attacks directed at the honeypots can be observed. It has ten extra features such as IDS Detection, Malware Detection, and Ashula Detection than previous available datasets which are useful in NIDS analyToward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization 109 sis and evaluation. The normal traffic here has been simulated repeatedly during the attacks and producing only DNS and mail traffic data, which is not reflected in real world normal network traffic, so there are no false positives, which are important for minimizing the number of alerts (Song et al., 2011) (M. Sato, 2012) (R. Chitrakar, 2012). Twente (University of Twente 2009): This dataset includes three services such as OpenSSH, Apache web server and Proftp using auth/ident on port 113 and captured data from a honeypot network by Netflow. There is some simultaneous network traffic such as auth/ident, ICMP, and IRC traffic, which are not completely benign or malicious. Moreover, this dataset contains some unknown and uncorrelated alerts traffic. It is labelled and is more realistic, but the lack of volume and diversity of attacks is obvious (Sperotto et al., 2009). UMASS (University of Massachusetts 2011): The dataset includes trace files, which are network packets, and some traces on wireless applications (of Massachusetts Amherst, 2011) (Nehinbe, 2011). It has been generated using a single TCP-based download request attack scenario. The dataset is not useful for testing IDS and IPS techniques due to the lack of variety of traffic and attacks (Swagatika Prusty and Liberatore, 2011). ISCX2012 (University of New Brunswick 2012): This dataset has two profiles, the Alpha-profile which carried out various multi-stage attack scenarios, and the Beta-profile, which is the benign traffic generator and generates realistic network traffic with background noise. It includes network traffic for HTTP, SMTP, SSH, IMAP,

POP3, and FTP protocols with full packet payload. However, it does not represent new network protocols since nearly 70% of todays network traffics are HTTPS and there are no HTTPS traces in this dataset. Moreover, the distribution of the simulated attacks is not based on real world statistics (Ali Shiravi and Ghorbani, 2012). ADFA (University of New South Wales 2013): This dataset includes normal training and validating data and 10 attacks per vector (Creech and Hu, 2013). It contains FTP and SSH password brute force, Java based Meterpreter, Add new Superuser, Linux Meterpreter payload and C100 Webshel attacks. In addition to the lack of attack diversity and variety of attacks, the behaviors of some attacks in this dataset are not well separated from the normal behavior (Xie and Hu, 2013) (Xie et al., 2014).

CHAPTER 3
SYSTEM ANALYSIS

3.1 Existing System

Blameless Bayes and Principal Component Analysis (PCA) were been used with the KDD99 dataset by Almansob and Lomte [9]. Similarly, PCA, SVM, and KDD99 were used Chithik and Rabbani for IDS [10]. In Aljawarneh et al's. Paper, their assessment and examinations were conveyed reliant on the NSL-KDD dataset for their IDS model [11] Composing inspects show that KDD99 dataset is continually used for IDS [6]–[10].There are 41 highlights in KDD99 and it was created in 1999. **Consequently, KDD99 is old and does not give any data about cutting edge new assault types**, example, multi day misuses and so forth. In this manner we utilized a cutting-edge and new CICIDS2017 dataset [12] in our investigation.

Limitations of existing system

- Strict Regulations
- Difficult to work with for non-technical users
- Restrictive to resources
- Constantly needs Patching
- Constantly being attacked

3.2 Proposed System

Important steps of the algorithm are given in below. 1) Normalization of every dataset. 2) Convert that dataset into the testing and training. 3) Form IDS models with the help of using RF, ANN, CNN and SVM algorithms. 4) Evaluate every model's performance

Advantages

- Protection from malicious attacks on your network.
- Deletion and/or guaranteeing malicious elements within a preexisting network.
- Prevents users from unauthorized access to the network.
- Deny's programs from certain resources that could be infected.

- Securing confidential information

3.3 Hardware and Software Requirements

SOFTWARE REQUIREMENTS

The functional requirements or the overall description documents include the product perspective and features, operating system and operating environment, graphics requirements, design constraints and user documentation.

The appropriation of requirements and implementation constraints gives the general overview of the project in regards to what the areas of strength and deficit are and how to tackle them.

- Programming language: Python 3.8 version
- Operating system: Windows

HARDWARE REQUIREMENTS

Minimum hardware requirements are very dependent on the particular software being developed by a given Enthought Python / Canopy / VS Code user. Applications that need to store large arrays/objects in memory will require more RAM, whereas applications that need to perform numerous calculations or tasks more quickly will require a faster processor.

- Operating system : windows, Linux
- Processor : minimum intel i3
- Ram : minimum 4 gb
- Hard disk : minimum 250gb

3.4 Analysis Model

3.4.1 STRUCTURE OF PROJECT

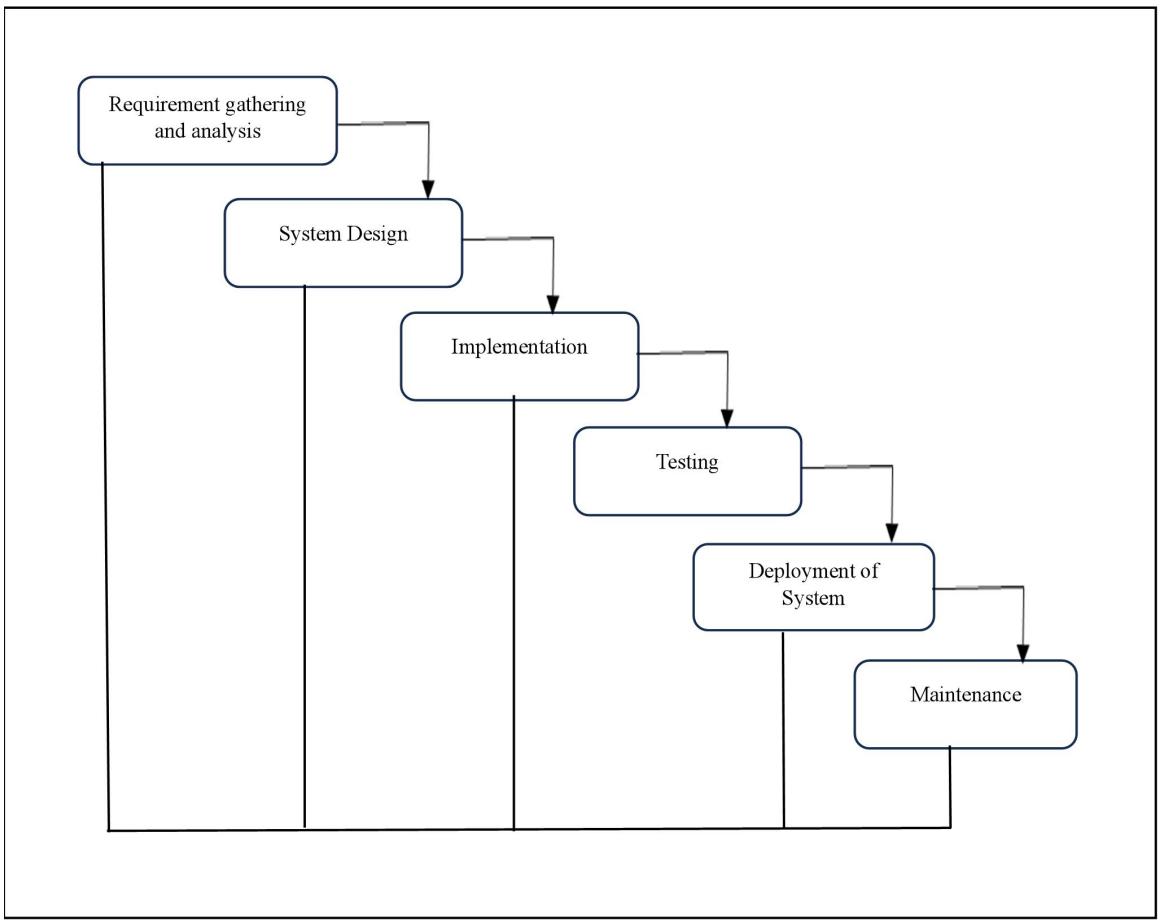


Fig: Project SDLC

- Project Requisites Accumulating and Analysis
- Application System Design
- Practical Implementation
- Manual Testing of My Application
- Application Deployment of System
- Maintenance of the Project

3.4.2 REQUISITES ACCUMULATING AND ANALYSIS

It's the first and foremost stage of the any project as our is a an academic leave for requisites amassing we followed of IEEE Journals and Amassed so many IEEE Relegated papers and final culled a Paper designated "Individual web revisitation by

setting and substance importance input and for analysis stage we took referees from the paper and did literature survey of some papers and amassed all the Requisites of the project in this stage

3.5 Input and Output

Input: The input of the system is the network traffic data collected from various sources, such as sensors, firewalls, routers, etc. The network traffic data consists of various attributes, such as packet size, protocol, source and destination IP addresses, ports, etc. The network traffic data can be in different formats, such as pcap, csv, txt, etc. The network traffic data can also be labeled as normal or anomalous, based on the ground truth or expert knowledge.

Output: The output of the system is the detection and classification of the network traffic data into normal or anomalous, based on the output of the machine learning techniques. The output can also include the confidence score of the classification, as well as the type of cyber attack detected, such as malware, ransomware, fraud, spoofing, etc. The output can also include the alerts and reports generated by the system for the system administrator and customer care. The alerts and reports can contain various information, such as alert type, alert message, alert time, report type, report content, report time, etc. The output can also include the network service provided to the customer, such as browsing, downloading, uploading, streaming, etc. The output can be in different formats, such as text, audio, video, graphical, etc.

CHAPTER 4
FEASIBILITY STUDY

4.1 Technical Feasibility

This aspect evaluates whether the system can be developed and deployed using the available resources, such as hardware, software, network, etc. The system requires a high-performance server to run the machine learning algorithms and store the network traffic data. The system also requires a reliable and secure network connection to collect and transmit the data from various sources. The system also requires various software tools and libraries to implement the machine learning techniques and the detection module. The system also requires a user-friendly interface to display the alerts and reports to the system administrator and the customer care. The system also requires a robust security mechanism to protect the data and the system from unauthorized access and modification. The system also requires a regular maintenance and update mechanism to ensure the optimal performance and accuracy of the system. The technical feasibility of the system is high, as all these requirements can be met using the existing or easily available resources.

4.2 Economical Feasibility

This aspect evaluates whether the system can be developed and deployed within the budget and time constraints, as well as whether the system can generate enough benefits to justify the costs. The system requires a significant initial investment to procure and install the hardware and software components, as well as to train and test the machine learning techniques. The system also requires a recurring cost to maintain and update the system, as well as to pay for the network service and electricity bills. However, the system can also generate substantial benefits, such as reducing the risk of cyber attacks, improving the network security and reliability, enhancing the customer satisfaction and loyalty, increasing the competitive advantage and reputation, etc. The economic feasibility of the system is moderate, as the benefits outweigh the costs in the long run.

4.3 Operational Feasibility

This aspect evaluates whether the system can be integrated and operated smoothly with the existing systems and processes, as well as whether the system can meet the expectations and needs of the users and stakeholders. The system can be integrated easily with the existing network infrastructure and devices, as it does not require any major changes or modifications. The system can also operate efficiently and effectively, as it can detect and classify the network traffic data into normal or anomalous, using various machine learning techniques. The system can also meet the expectations and needs of the users and stakeholders, such as providing a secure and reliable network service to the customers, providing support and assistance to the customers in case of any issues, providing alerts and reports to the system administrator and customer care in case of any cyber attack detection, etc. The operational feasibility of the system is high, as it can improve the quality and productivity of the network service.

CHAPTER 5

SOFTWARE REQUIREMENT SPECIFICATION

5.1 Functional Requirements

- Data Collection
- Data Preprocessing
- Training And Testing
- Modelling
- Predicting

1. Data Collection:

This requirement focuses on collecting relevant data for cyber attack detection. It involves determining the sources of data, such as network traffic, system logs, user behavior, or threat intelligence feeds. The system should be capable of efficiently gathering and integrating data from these sources for further analysis and processing.

2. Data Preprocessing:

Data preprocessing refers to the steps taken to clean, transform, and prepare the collected data for analysis. It includes tasks such as data cleaning to remove noise and outliers, data normalization for consistent scaling, feature extraction to identify relevant information, and handling missing data. The system should be able to perform these preprocessing tasks effectively to ensure accurate and reliable results.

3. Training and Testing:

The system needs to have the functionality to train machine learning models using labeled data. This involves selecting appropriate algorithms, setting up the training process, and feeding the prepared data into the models for learning. Additionally, the system should support testing the trained models using separate testing datasets to assess their performance, such as accuracy, precision, recall, or F1-score.

4. Modeling:

Modeling refers to the creation and optimization of machine learning models for cyber attack detection. This requirement includes tasks such as selecting suitable algorithms (e.g., decision trees, support vector machines, neural networks) based on the project's objectives and requirements. The system should provide the necessary tools and techniques to train, tune, and optimize the models to achieve high detection accuracy and minimize false positives or false negatives.

5. Predicting:

Once the machine learning models are trained and validated, the system should enable real-time or near real-time prediction of cyber attacks. It involves applying the trained models to incoming data streams or batch data to detect and classify potential attacks. The system should provide mechanisms for efficient and timely prediction, such as streaming analytics or batch processing, depending on the project's requirements.

5.2 Non Functional Requirements

NON-FUNCTIONAL REQUIREMENT (NFR) specifies the quality attribute of a software system. They judge the software system based on Responsiveness, Usability, Security, Portability and other non-functional standards that are critical to the success of the software system. Example of nonfunctional requirement, *"how fast does the website load?"* Failing to meet non-functional requirements can result in systems that fail to satisfy user needs. Non- functional Requirements allows you to impose constraints or restrictions on the design of the system across the various agile backlogs. Example, the site should load in 3 seconds when the number of simultaneous users are > 10000. Description of non-functional requirements is just as critical as a functional requirement.

- Usability requirement
- Serviceability requirement
- Manageability requirement

- Recoverability requirement
- Security requirement
- Data Integrity requirement
- Capacity requirement
- Availability requirement
- Scalability requirement
- Interoperability requirement
- Reliability requirement
- Maintainability requirement
- Regulatory requirement
- Environmental requirement

EXAMPLES OF NON-FUNCTIONALREQUIREMENTS

Here, are some examples of non-functional requirement:

- Users must upload dataset
- The software should be portable. So, moving from one OS to other OS does not create any problem.
- Privacy of information, the export of restricted technologies, intellectual property rights, etc. should be audited.

ADVANTAGES OF NON-FUNCTIONALREQUIREMENT

Benefits/pros of Non-functional testing are:

- The nonfunctional requirements ensure the software system follow legal and compliance rules.
- They ensure the reliability, availability, and performance of the software system
- They ensure good user experience and ease of operating the software.
- They help in formulating security policy of the software system.

DISADVANTAGES OF NON-FUNCTIONALREQUIREMENT

Cons/drawbacks of Non-function requirement are:

- None functional requirement may affect the various high-level software

subsystem

- They require special consideration during the software architecture/high-level design phase which increases costs.
- Their implementation does not usually map to the specific software sub-system, It is tough to modify non-functional once you pass the architecture phase.

CHAPTER 6

SYSTEM DESIGN

6.1 Introduction

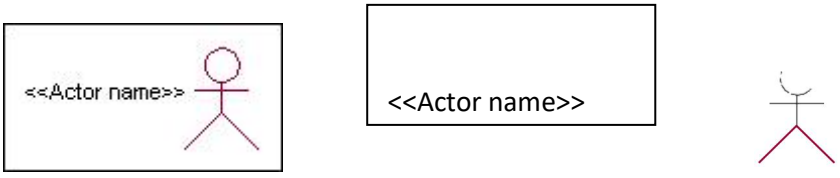
In System Design has divided into three types like GUI Designing, UML Designing with avails in development of project in facile way with different actor and its utilizer case by utilizer case diagram, flow of the project utilizing sequence, Class diagram gives information about different class in the project with methods that have to be utilized in the project if comes to our project our UML Will utilizable in this way The third and post import for the project in system design is Data base design where we endeavor to design data base predicated on the number of modules in our project.

The System Design Document describes the system requirements, operating environment, system and subsystem architecture, files and database design, input formats, output layouts, human-machine interfaces, detailed design, processing logic, and external interfaces.

Global Use Case Diagrams:

Identification of actors:

Actor: Actor represents the role a user plays with respect to the system. An actor interacts with, but has no control over the use cases.
Graphical representation:



Actor

- An actor is someone or something that:
- Interacts with or uses the system.
- Provides input to and receives information from the system.

- Is external to the system and has no control over the use cases. Actors are discovered by examining:
- Who directly uses the system?
- Who is responsible for maintaining the system?
- External hardware used by the system.
- Other systems that need to interact with the system. Questions to identify actors:
- Who is using the system? Or, who is affected by the system? Or, which groups need help from the system to perform at ask?
- Who affects the system? Or, which user groups are needed by the system to perform its functions? These functions can be both main functions and secondary functions such as administration.
- Which external hardware or systems (if any) use the system to perform tasks?
- What problems does this application solve (that is, for whom)?
- And, finally, how do users use the system (use case)? What are they doing with the system?

The actors identified in this system are:

a) **System Administrator**

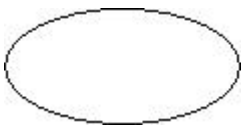
b) **Customer**

c) **Customer Care**

Identification of use cases:

Usecase: A use case can be described as a specific way of using the system from a user's (actor's)perspective.

Graphical representation:



A more detailed description might characterize a use case as:

- Pattern of behavior the system exhibits
- A sequence of related transactions performed by an actor and the system
- Delivering something of value to the actor Use cases provide a means to:
- capture system requirements
- communicate with the end users and domain experts
- test the system

Use cases are best discovered by examining the actors and defining what the actor will be able to do with the system.
Guide lines for identifying use cases:

- For each actor, find the tasks and functions that the actor should be able to perform or that the system needs the actor to perform. The use case should represent a course of events that leads to clear goal
- Name the use cases.
- Describe the use cases briefly by applying terms with which the user is familiar. This makes the description less ambiguous
- Questions to identify use cases:
- What are the tasks of each actor?
- Will any actor create, store, change, remove or read information in the system?
- What use case will store, change, remove or read this information?
- Will any actor need to inform the system about sudden external changes?
- Does any actor need to inform about certain occurrences in the system?
- What use cases will support and maintains the system?

Flow of Events

A flow of events is a sequence of transactions (or events) performed by the system. They typically contain very detailed information, written in terms of what the system should do, not how the system accomplishes the task. Flow of events are created as separate files or documents in your favorite text editor and then attached or linked to a use case using the Files tab of a model element.
A flow of events should include:

- When and how the use case starts and ends

- Use case/actor interactions
- Data needed by the use case
- Normal sequence of events for the use case
- Alternate or exceptional flows Construction of Use case diagrams:

Use-case diagrams graphically depict system behavior (use cases). These diagrams present a high level view of how the system is used as viewed from an outsider's (actor's) perspective. A use-case diagram may depict all or some of the use cases of a system.

A use-case diagram can contain:

- actors ("things" outside the system)
- use cases (system boundaries identifying what the system should do)
- Interactions or relationships between actors and use cases in the system including the associations, dependencies, and generalizations.

Relationships in use cases:

1. Communication:

The communication relationship of an actor in a use case is shown by connecting the actor symbol to the use case symbol with a solid path. The actor is said to communicate with the use case.

2. Uses:

A Uses relationship between the use cases is shown by generalization arrow from the use case.

3. Extends:

The extend relationship is used when we have one use case that is similar to another use case but does a bit more. In essence it is like subclass.

SEQUENCE DIAGRAMS

A sequence diagram is a graphical view of a scenario that shows object interaction in a time- based sequence what happens first, what happens next. Sequence diagrams establish the roles of objects and help provide essential information to determine class responsibilities and interfaces. There are two main differences between sequence and collaboration diagrams: sequence diagrams show time-based object interaction while

collaboration diagrams show how objects associate with each other. A sequence diagram has two dimensions: typically, vertical placement represents time and horizontal placement represents different objects.

Object:

An object has state, behavior, and identity. The structure and behavior of similar objects are defined in their common class. Each object in a diagram indicates some instance of a class. An object that is not named is referred to as a class instance.
The object icon is similar to a class icon except that the name is underlined: An object's concurrency is defined by the concurrency of its class.

Message:

A message is the communication carried between two objects that trigger an event. A message carries information from the source focus of control to the destination focus of control. The synchronization of a message can be modified through the message specification. Synchronization means a message where the sending object pauses to wait for results.

Link:

A link should exist between two objects, including class utilities, only if there is a relationship between their corresponding classes. The existence of a relationship between two classes symbolizes a path of communication between instances of the classes: one object may send messages to another. The link is depicted as a straight line between objects or objects and class instances in a collaboration diagram. If an object links to itself, use the loop version of the icon.

CLASS DIAGRAM:

Identification of analysis classes:

A class is a set of objects that share a common structure and common behavior (the same attributes, operations, relationships and semantics). A class is an abstraction of real-world items. There are 4 approaches for identifying classes:

a. Noun phrase approach:
b. Common class pattern approach.
c. Use case Driven Sequence or Collaboration approach.
d. Classes, Responsibilities and collaborators Approach

1. Noun Phrase Approach:

The guidelines for identifying the classes:

- Look for nouns and noun phrases in the use cases.
- Some classes are implicit or taken from general knowledge.
- All classes must make sense in the application domain; Avoid computer implementation classes – defer them to the design stage.
- Carefully choose and define the class names After identifying the classes we have to eliminate the following types of classes:
- Adjective classes.

2. Common class pattern approach:

The following are the patterns for finding the candidate classes:

- Concept class.
- Events class.
- Organization class
- Peoples class
- Places class
- Tangible things and devices class.

3. Use case driven approach:

We have to draw the sequence diagram or collaboration diagram. If there is need for some classes to represent some functionality then add new classes which perform those functionalities.

4. CRC approach:

The process consists of the following steps:

- Identify classes' responsibilities (and identify the classes)
- Assign the responsibilities
- Identify the collaborators. Identification of responsibilities of each class:

The questions that should be answered to identify the attributes and methods of a class respectively are:

a. What information about an object should we keep track of?
b. What services must a class provide? Identification of relationships among the

classes:

Three types of relationships among the objects are:

Association: How objects are associated?

Super-sub structure: How are objects organized into super classes and sub classes?
Aggregation: What is the composition of the complex classes?
Association:

The **questions** that will help us to identify the associations are:

a. Is the class capable of fulfilling the required task by itself?

b. If not, what does it need?

c. From what other classes can it acquire what it needs? Guidelines for identifying the tentative associations:

- A dependency between two or more classes may be an association. Association often corresponds to a verb or prepositional phrase.
- A reference from one class to another is an association. Some associations are implicit or taken from general knowledge.

Some common association patterns are:

Location association like part of, next to, contained in…. Communication association like talk to, order to ……
We have to eliminate the unnecessary association like implementation associations, ternary or n- ary associations and derived associations.
Super-sub class relationships:

Super-sub class hierarchy is a relationship between classes where one class is the parent class of another class (derived class). This is based on inheritance.
Guidelines for identifying the super-sub relationship, a generalization are

1. **Top-down*:***

Look for noun phrases composed of various adjectives in a class name. Avoid excessive refinement. Specialize only when the sub classes have significant behavior.

2. **Bottom-up*:***

Look for classes with similar attributes or methods. Group them by moving the

common attributes and methods to an abstract class. You may have to alter the definitions a bit.

3. **Reusability*:***

Move the attributes and methods as high as possible in the hierarchy.

4. **Multiple inheritances*:***

Avoid excessive use of multiple inheritances. One way of getting benefits of multiple inheritances is to inherit from the most appropriate class and add an object of another class as an attribute.

Aggregation or a-part-of relationship:

It represents the situation where a class consists of several component classes. A class that is composed of other classes doesn't behave like its parts. It behaves very difficultly. The major properties of this relationship are transitivity and antisymmetric. The **questions** whose answers will determine the distinction between the part and whole relationships are:

- Does the part class belong to the problem domain?
- Is the part class within the system's responsibilities?
- Does the part class capture more than a single value? (If not then simply include it as an attribute of the whole class)
- Does it provide a useful abstraction in dealing with the problem domain?

There are three types of aggregation relationships.

They are:

Assembly:

It is constructed from its parts and an assembly-part situation physically exists.

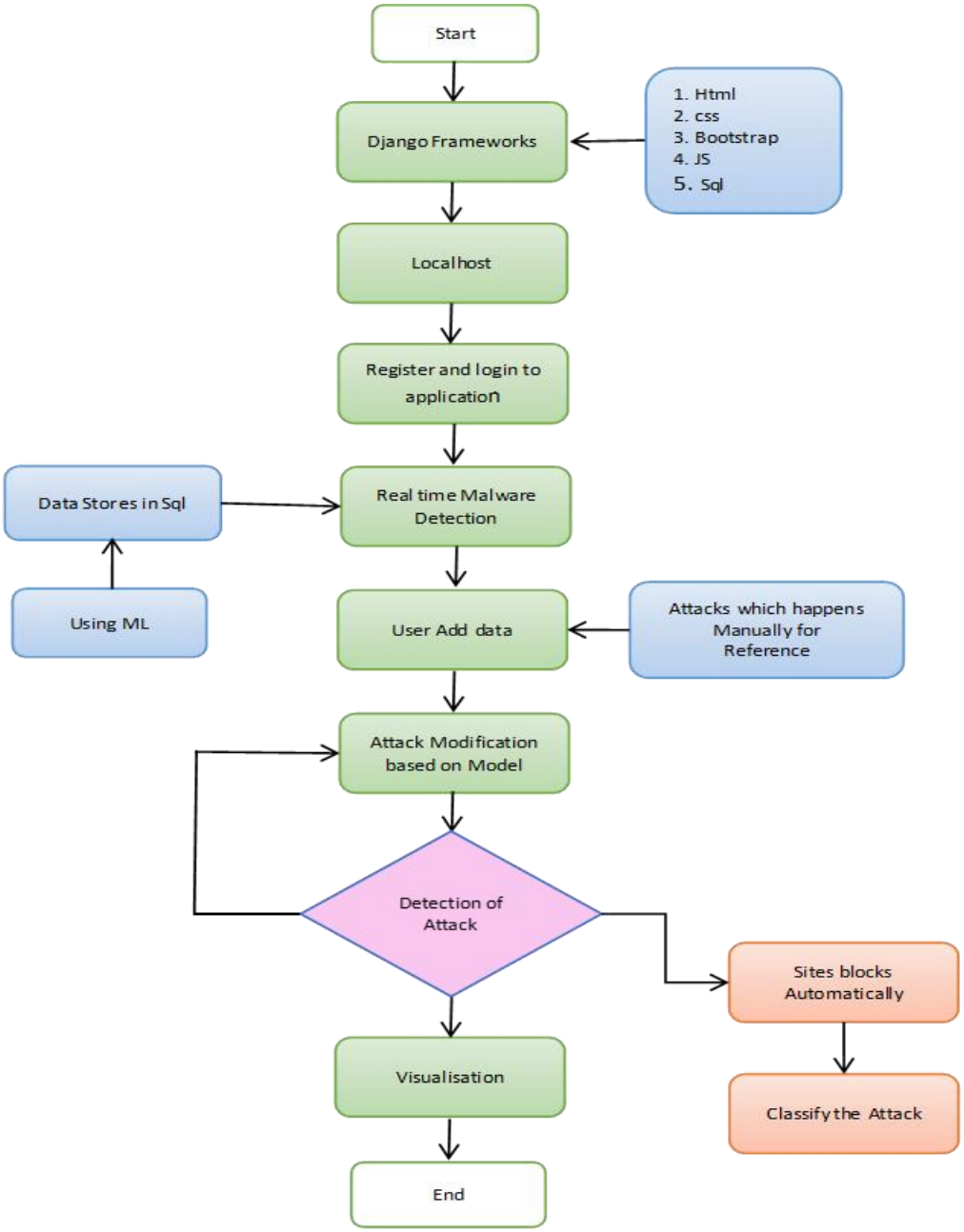
Container:

A physical whole encompasses but is not constructed from physical parts.

Collection member:

A conceptual whole encompasses parts that may be physical or conceptual. The container and collection are represented by hollow diamonds but composition is represented by solid diamond.

6.2 Data Flow Diagrams



6.3 UML Diagrams

The System Design Document describes the system requirements, operating environment, system and subsystem architecture, files and database design, input formats, output layouts, human-machine interfaces, detailed design, processing logic, and external interfaces.

6.3.1 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

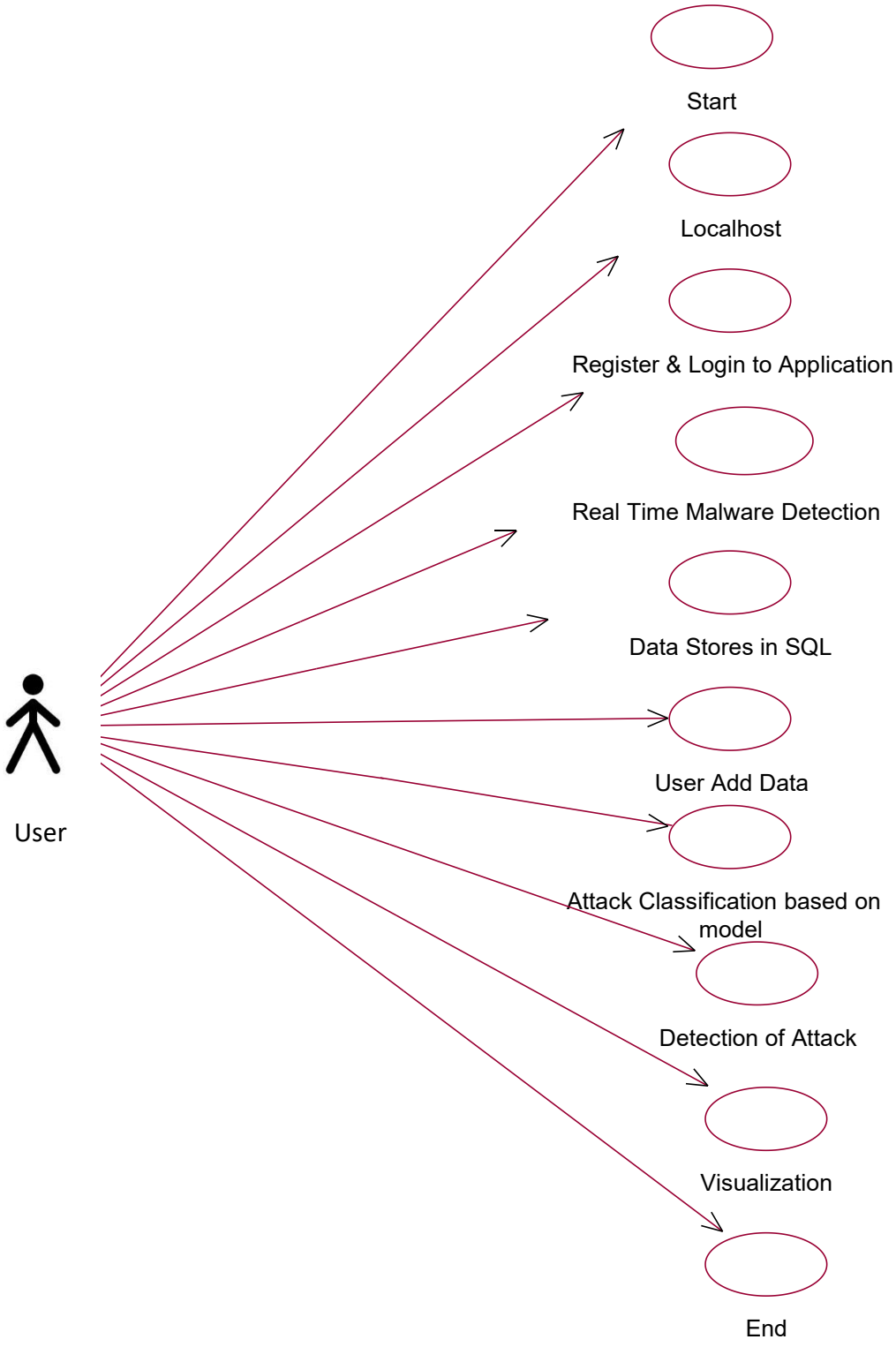


Fig : Use Case Diagram

6.3.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

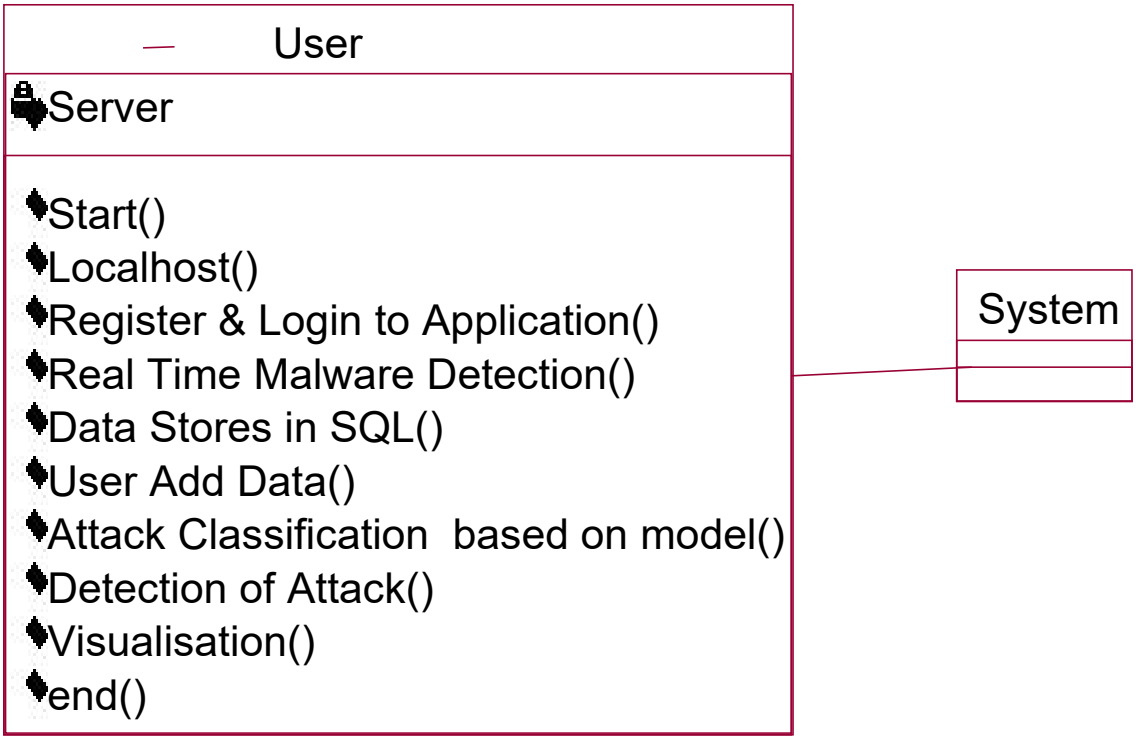


Fig : Class Diagram

6.3.3 SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

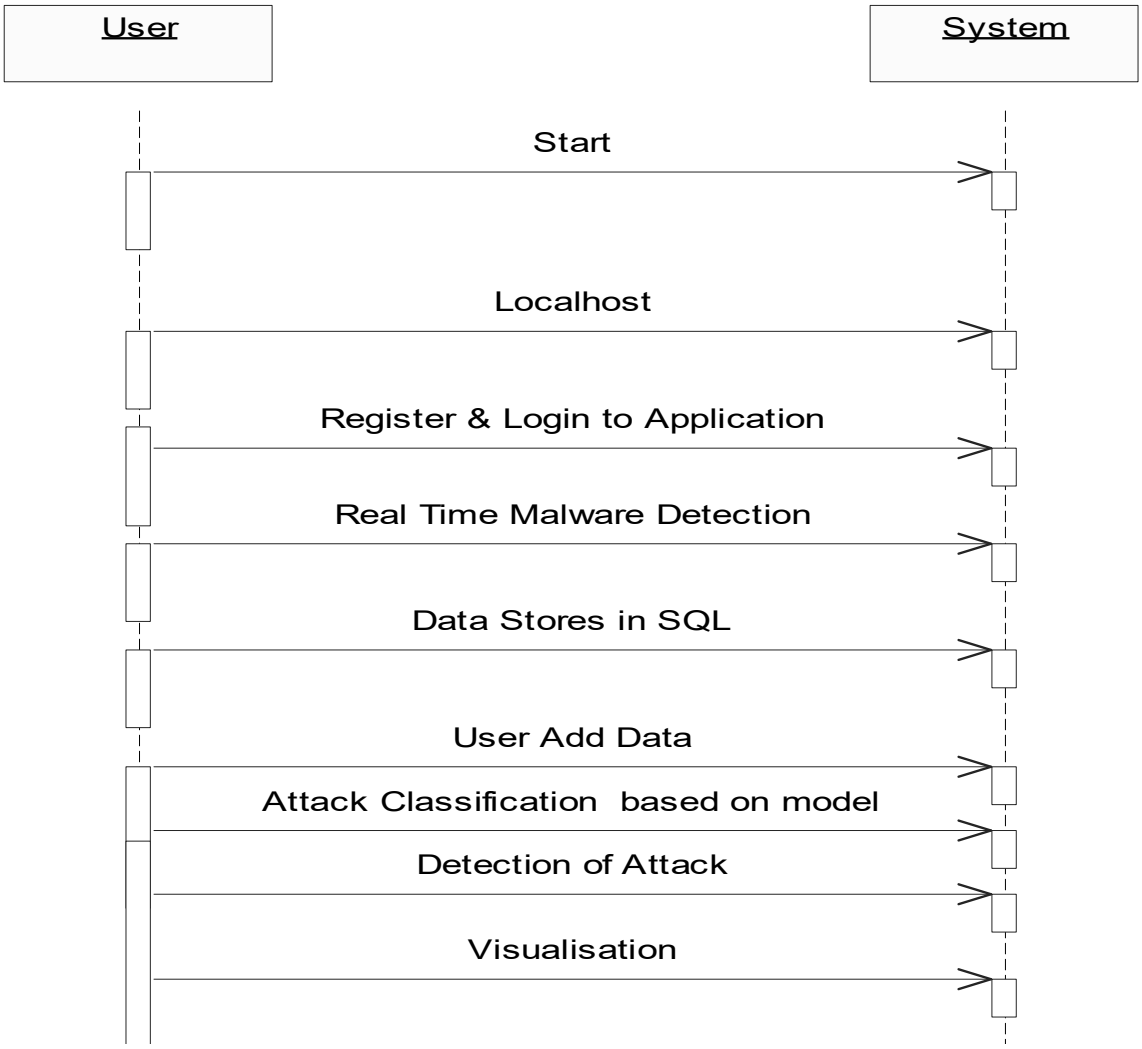


Fig : Sequence Diagram

6.5 Sample Code

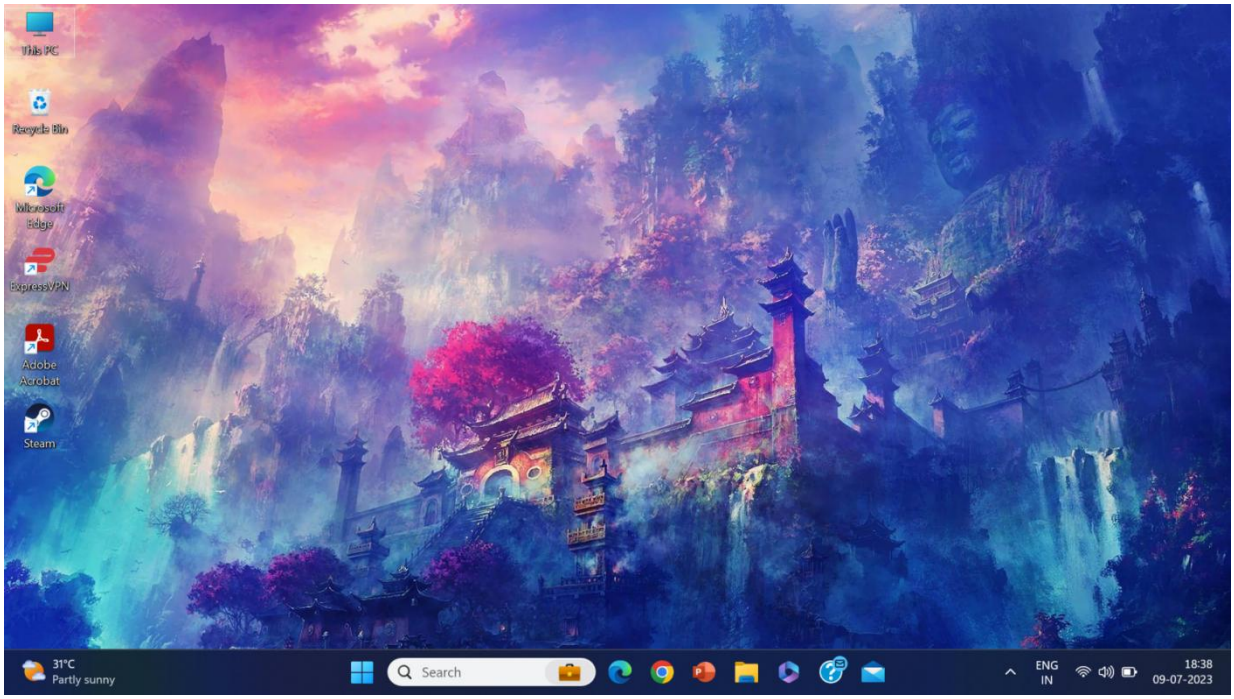
```
main = tkinter.Tk()

main.title("Android Malware Detection")

main.geometry("1300x1200")

global filename

global train

global svm_acc, nn_acc, svmga_acc, annga_acc

global X_train, X_test, y_train, y_test

global svmga_classifier

global nnga_classifier

global svm_time,svmga_time,nn_time,nnga_time


def upload():

    global filename

    filename = filedialog.askopenfilename(initialdir="dataset")

    pathlabel.config(text=filename)

    text.delete('1.0', END)

    text.insert(END,filename+" loaded\n");


def generateModel():

    global X_train, X_test, y_train, y_test

    text.delete('1.0', END)

    train = pd.read_csv(filename)
```

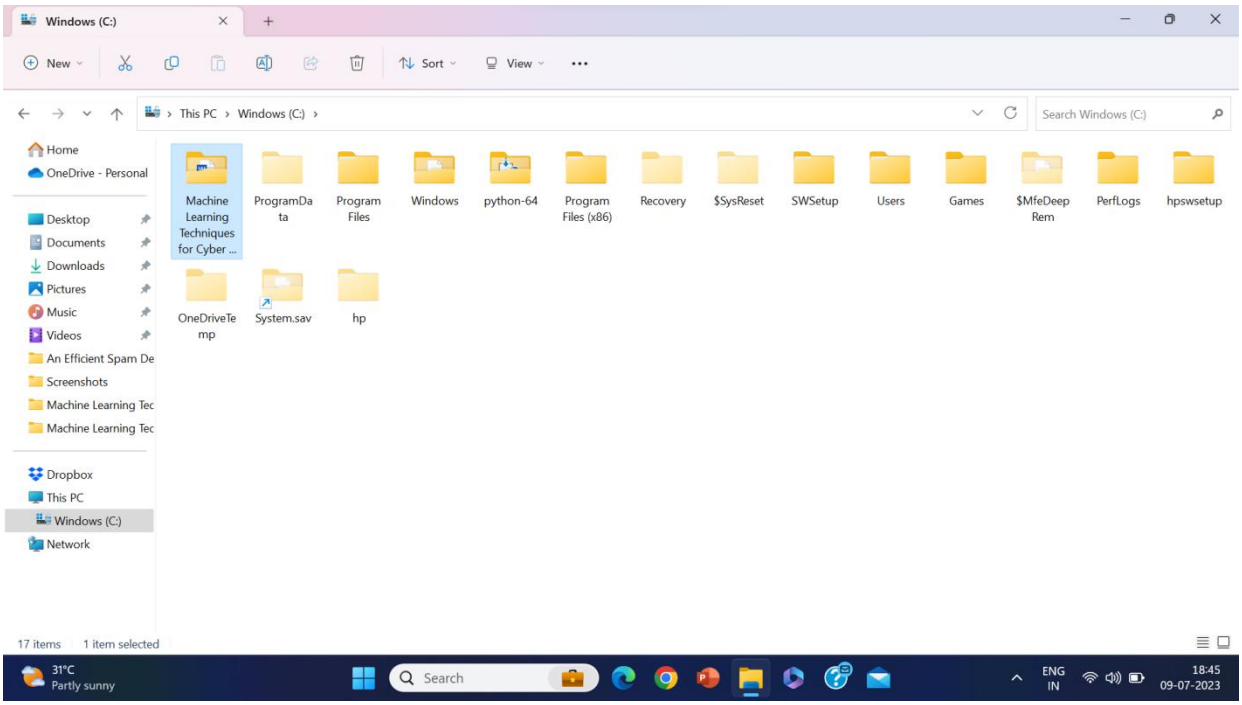
```
    rows = train.shape[0]  # gives number of row count
    cols = train.shape[1]  # gives number of col count
    features = cols - 1
    print(features)
    X = train.values[:, 0:features]
    Y = train.values[:, features]
    print(Y)
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size = 0.2,
random_state = 0)
    text.insert(END,"Dataset Length : "+str(len(X))+"\n");
    text.insert(END,"Splitted Training Length : "+str(len(X_train))+"\n");
    text.insert(END,"Splitted Test Length : "+str(len(X_test))+"\n\n");


def prediction(X_test, cls):  #prediction done here
    y_pred = cls.predict(X_test)
    for i in range(len(X_test)):
      print("X=%s, Predicted=%s" % (X_test[i], y_pred[i]))
    return y_pred


# Function to calculate accuracy
def cal_accuracy(y_test, y_pred, details):
    cm = confusion_matrix(y_test, y_pred)
    accuracy = accuracy_score(y_test,y_pred)*100
```

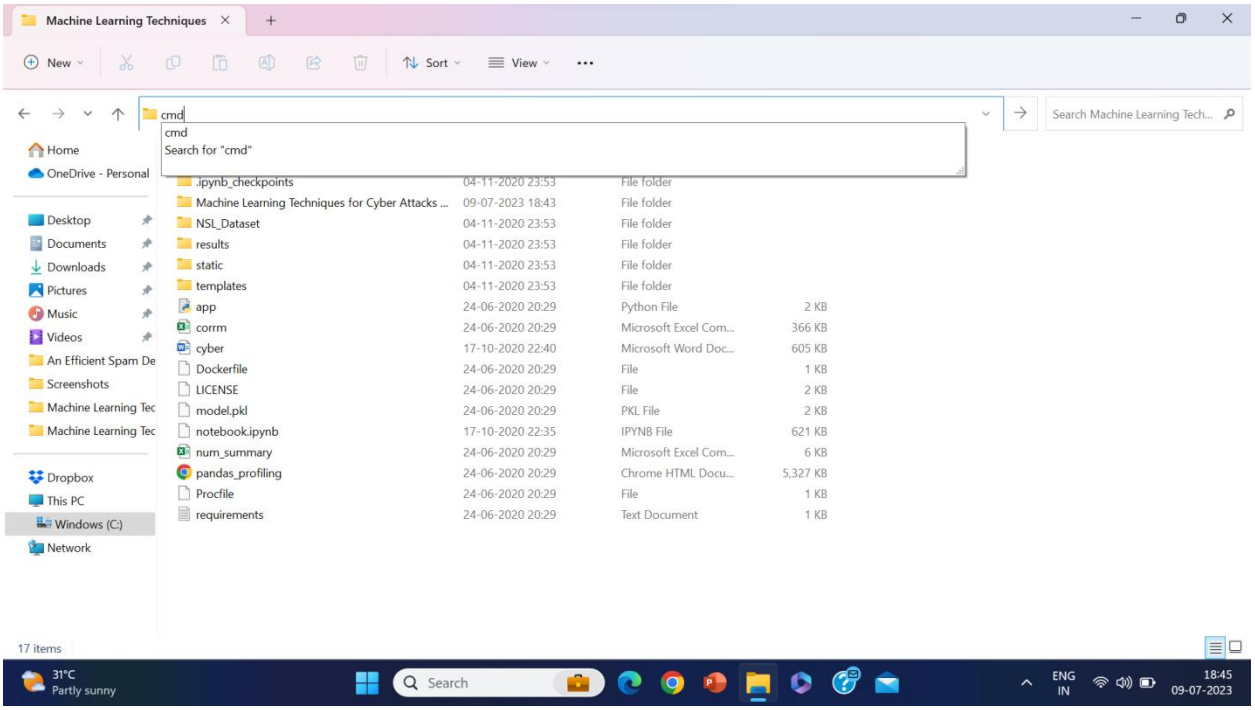
```
text.insert(END,details+"\n\n")

text.insert(END,"Accuracy : "+str(accuracy)+"\n\n")

text.insert(END,"Report : "+str(classification_report(y_test, y_pred))+"\n")

text.insert(END,"Confusion Matrix : "+str(cm)+"\n\n\n\n\n")

return accuracy
```

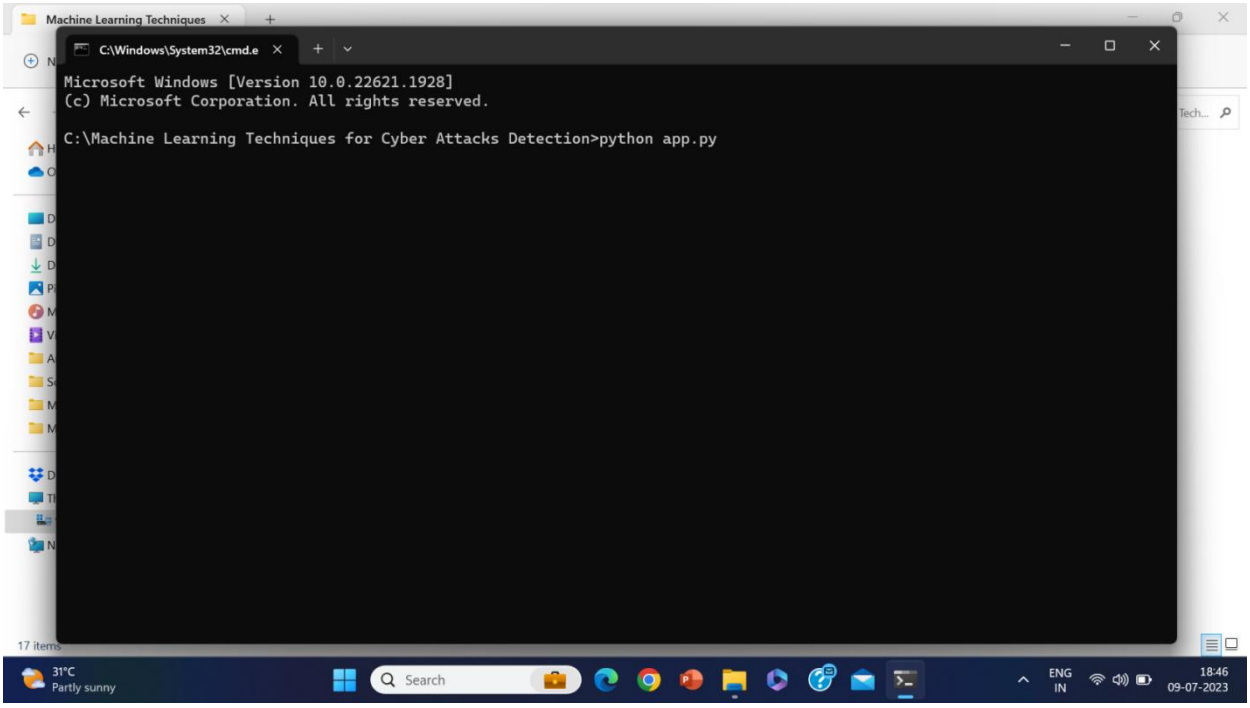
CHAPTER 7

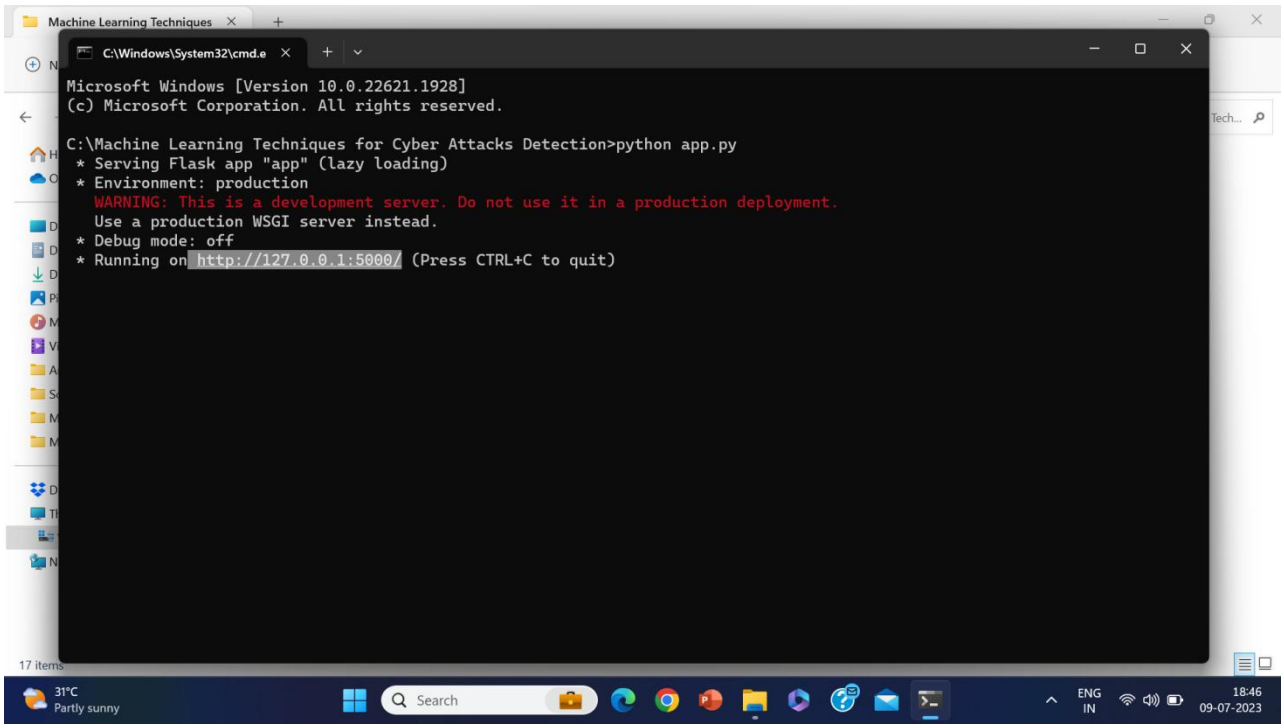
SCREEN SHOTS

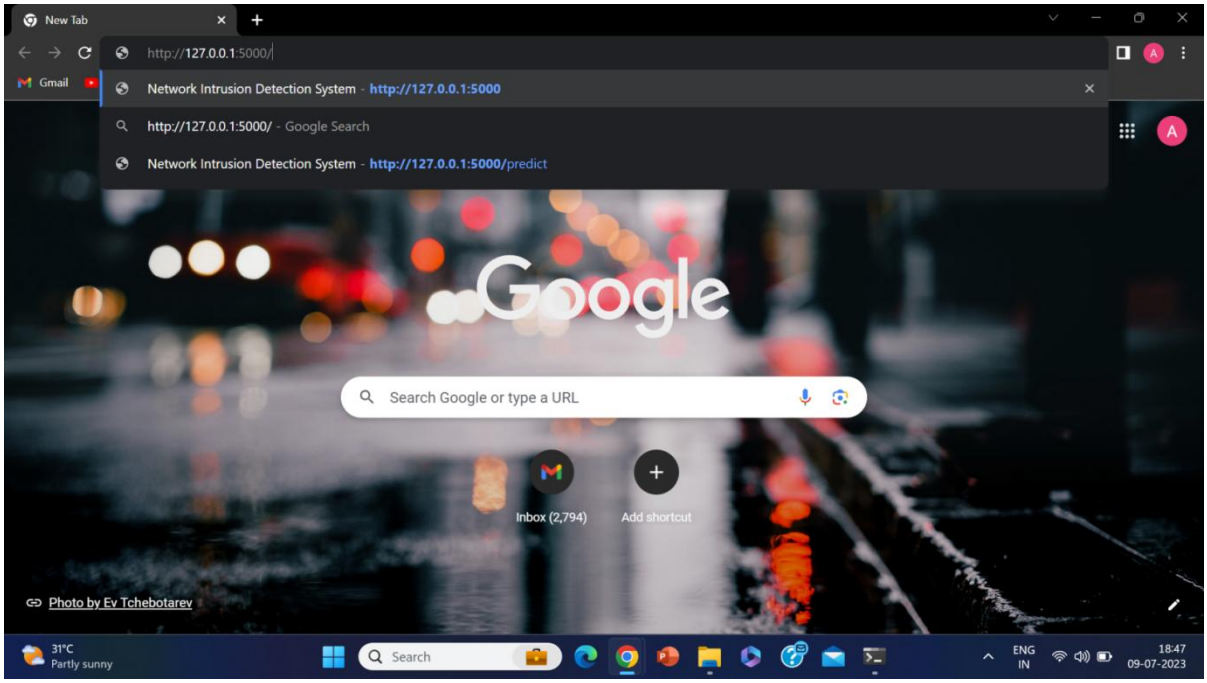


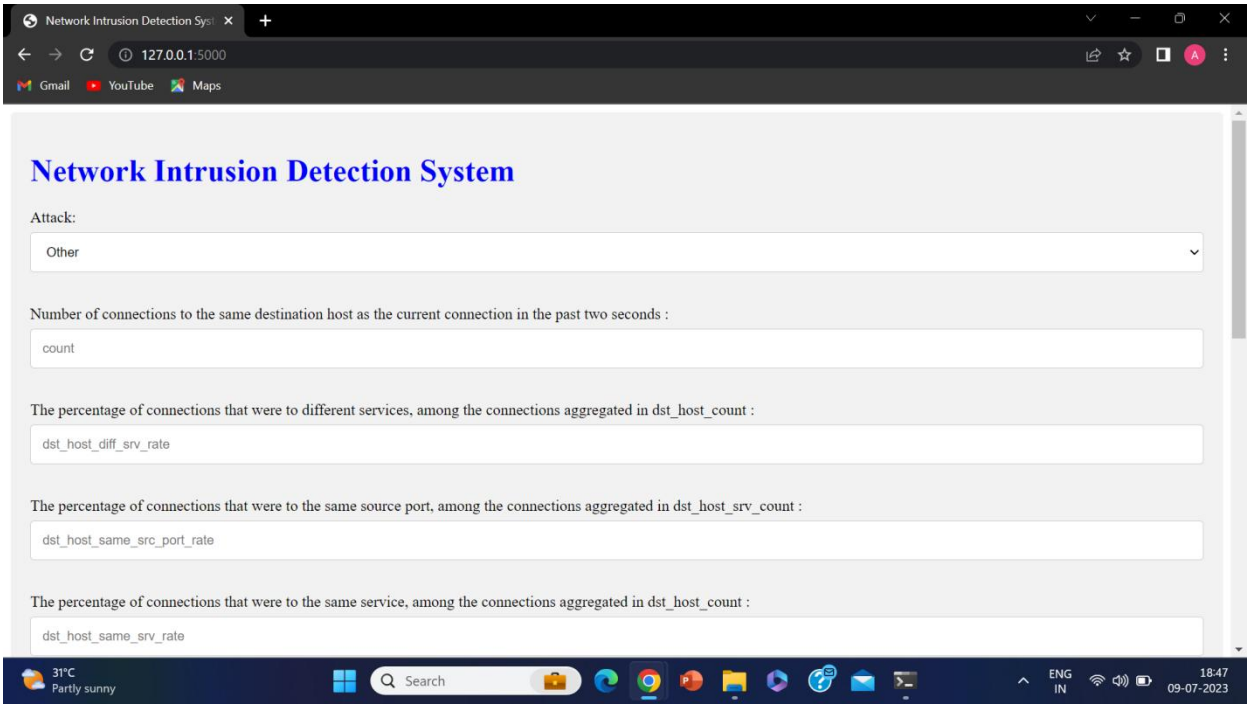


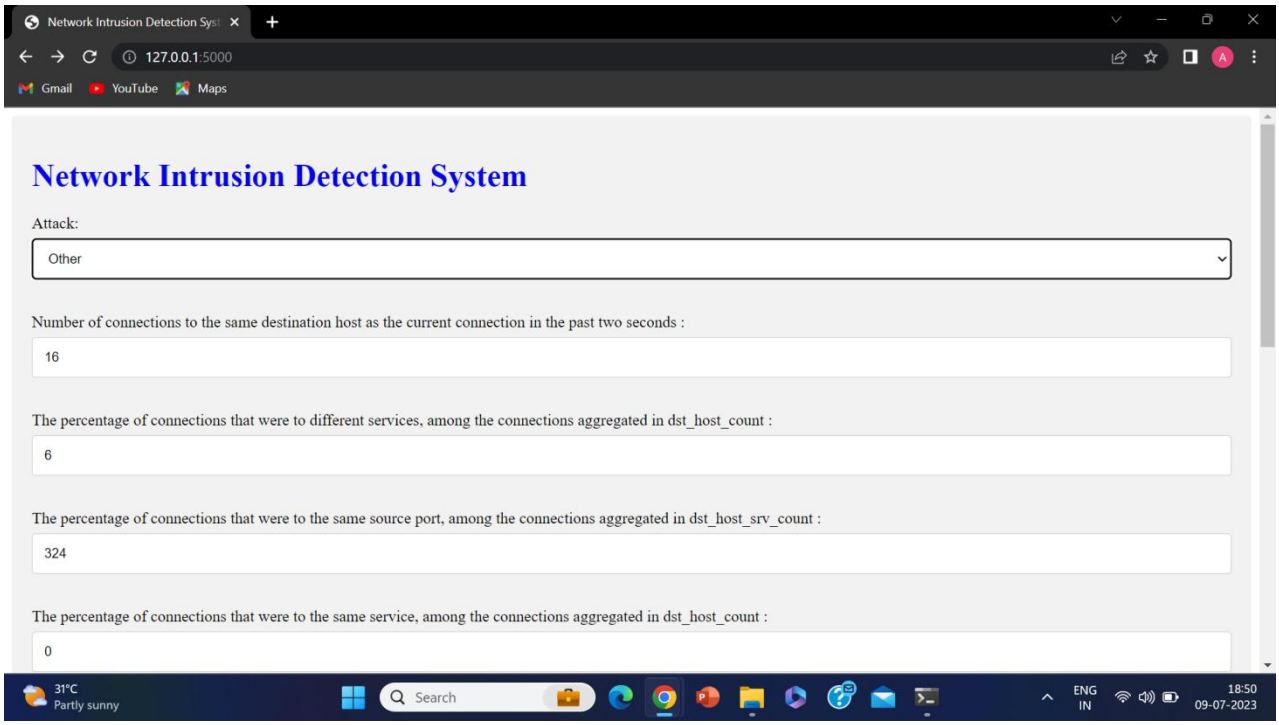

Machine Learning Techniques
New
Sort
View
cmd
cmd
Search for "cmd"
Search Machine Learning Tech...
Home
OneDrive - Personal
Desktop
Documents
Downloads
Pictures
Music
Videos
An Efficient Spam De
Screenshots
Machine Learning Tec
Machine Learning Tec
Dropbox
This PC
Windows (C:)
Network
.ipynb_checkpoints 04-11-2020 23:53 File folder
Machine Learning Techniques for Cyber Attacks ... 09-07-2023 18:43 File folder
NSL_Dataset 04-11-2020 23:53 File folder
results 04-11-2020 23:53 File folder
static 04-11-2020 23:53 File folder
templates 04-11-2020 23:53 File folder
app 24-06-2020 20:29 Python File 2 KB
corrm 24-06-2020 20:29 Microsoft Excel Com... 366 KB
cyber 17-10-2020 22:40 Microsoft Word Doc... 605 KB
Dockerfile 24-06-2020 20:29 File 1 KB
LICENSE 24-06-2020 20:29 File 2 KB
model.pkl 24-06-2020 20:29 PKL File 2 KB
notebook.ipynb 17-10-2020 22:35 IPYNB File 621 KB
num_summary 24-06-2020 20:29 Microsoft Excel Com... 6 KB
pandas_profiling 24-06-2020 20:29 Chrome HTML Docu... 5,327 KB
Procfile 24-06-2020 20:29 File 1 KB
requirements 24-06-2020 20:29 Text Document 1 KB
17 items
31°C
Partly sunny
Search
ENG
IN
18:45
09-07-2023

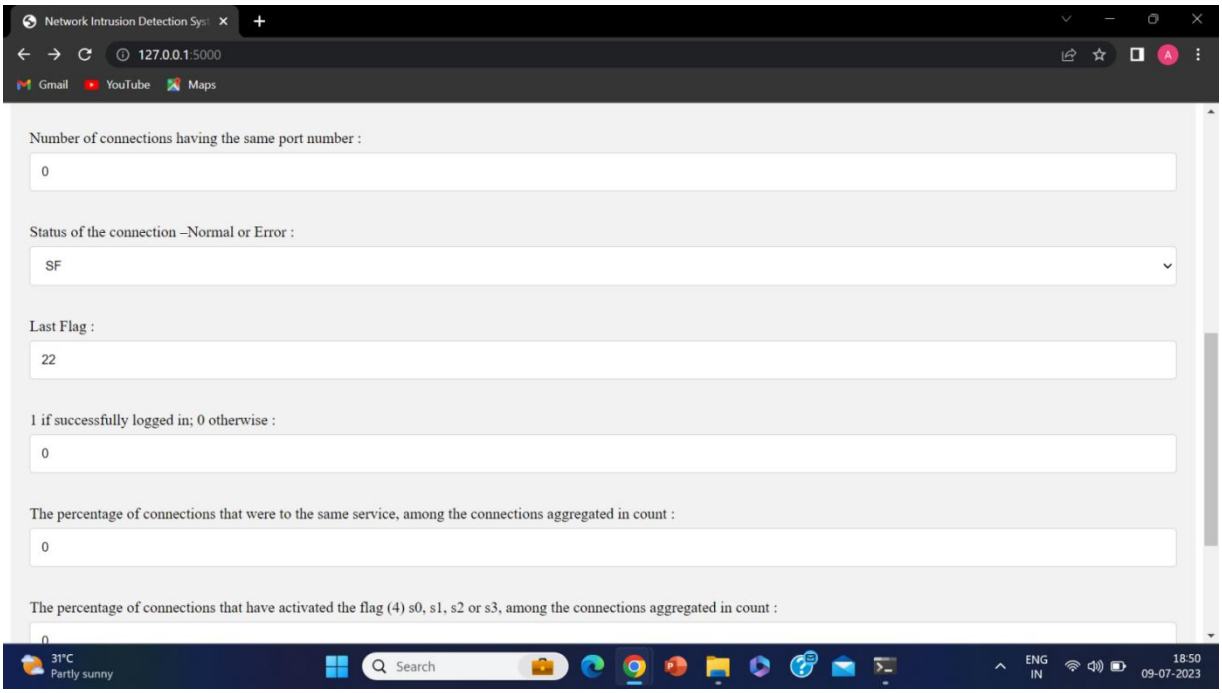

Machine Learning Techniques
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22621.1928]
(c) Microsoft Corporation. All rights reserved.
C:\Machine Learning Techniques for Cyber Attacks Detection>python app.py
17 items
31°C
Partly sunny
Search
ENG
IN
18:46
09-07-2023

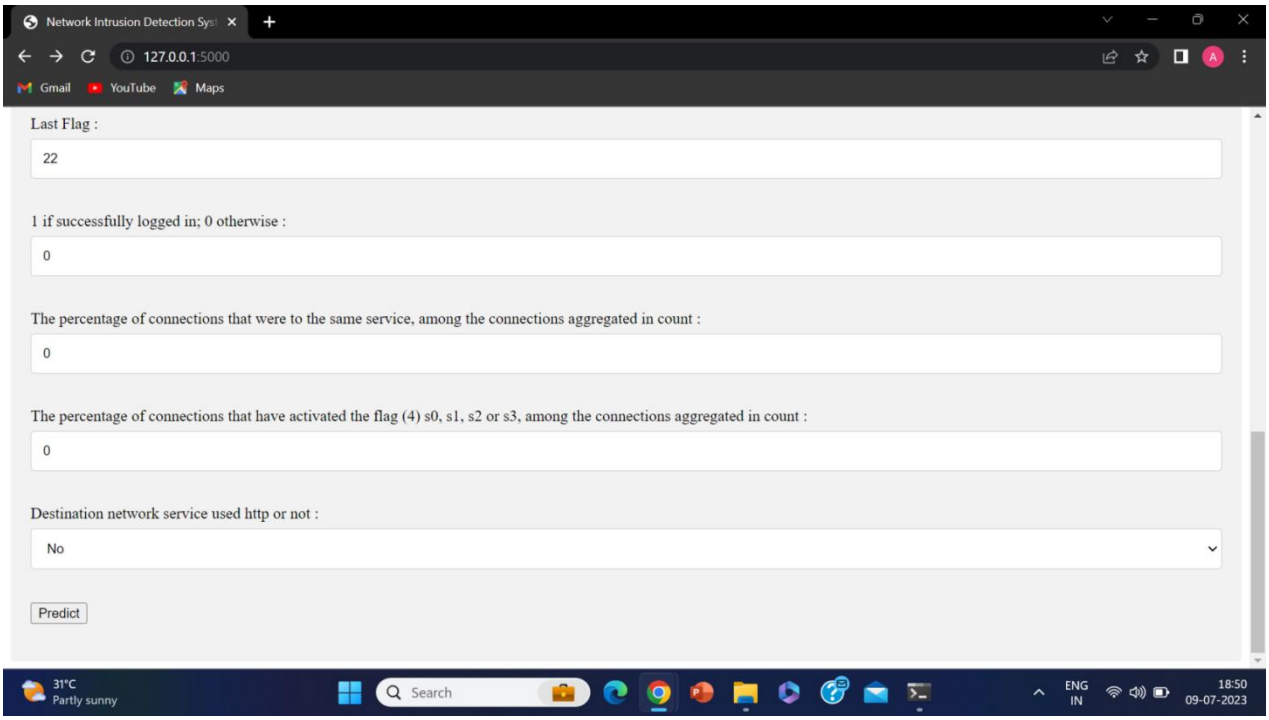

Machine Learning Techniques
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22621.1928]
(c) Microsoft Corporation. All rights reserved.
C:\Machine Learning Techniques for Cyber Attacks Detection>python app.py
* Serving Flask app "app" (lazy loading)
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
17 items
31°C
Partly sunny
Search
ENG
IN
18:46
09-07-2023

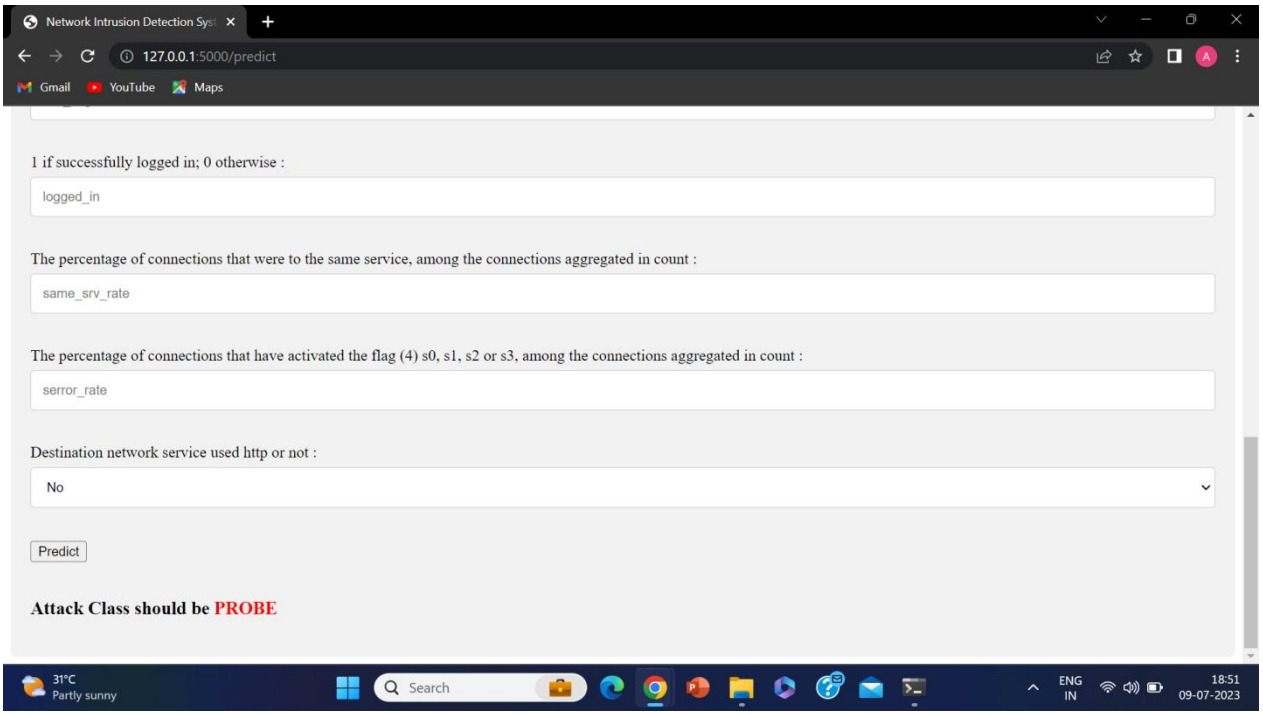

New Tab
http://127.0.0.1:5000/
Gmail
Network Intrusion Detection System - http://127.0.0.1:5000
http://127.0.0.1:5000/ - Google Search
Network Intrusion Detection System - http://127.0.0.1:5000/predict
Google
Search Google or type a URL
Inbox (2,794)
Add shortcut
Photo by Ev Tchebotarev
31°C
Partly sunny
Search
ENG
IN
18:47
09-07-2023

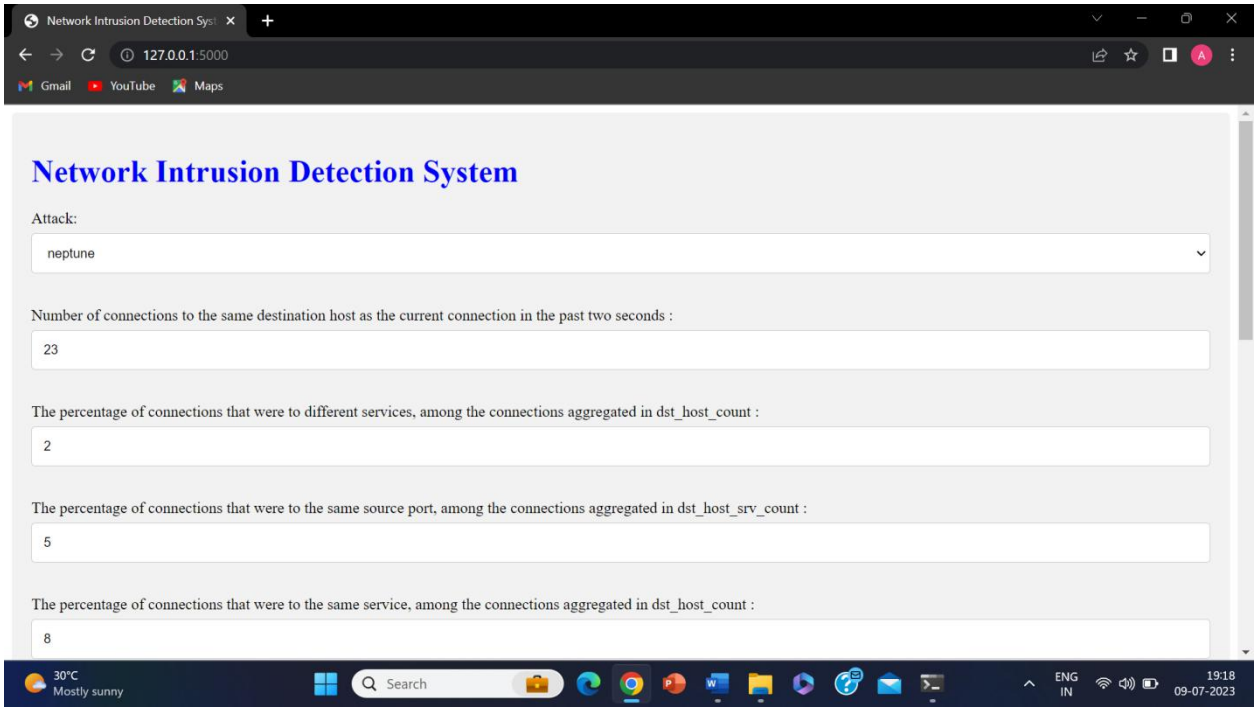

Network Intrusion Detection Syst
127.0.0.1:5000
Gmail
YouTube
Maps
Network Intrusion Detection System
Attack:
Other
Number of connections to the same destination host as the current connection in the past two seconds :
count
The percentage of connections that were to different services, among the connections aggregated in dst_host_count :
dst_host_diff_srv_rate
The percentage of connections that were to the same source port, among the connections aggregated in dst_host_srv_count :
dst_host_same_src_port_rate
The percentage of connections that were to the same service, among the connections aggregated in dst_host_count :
dst_host_same_srv_rate
31°C
Partly sunny
Search
ENG
IN
18:47
09-07-2023

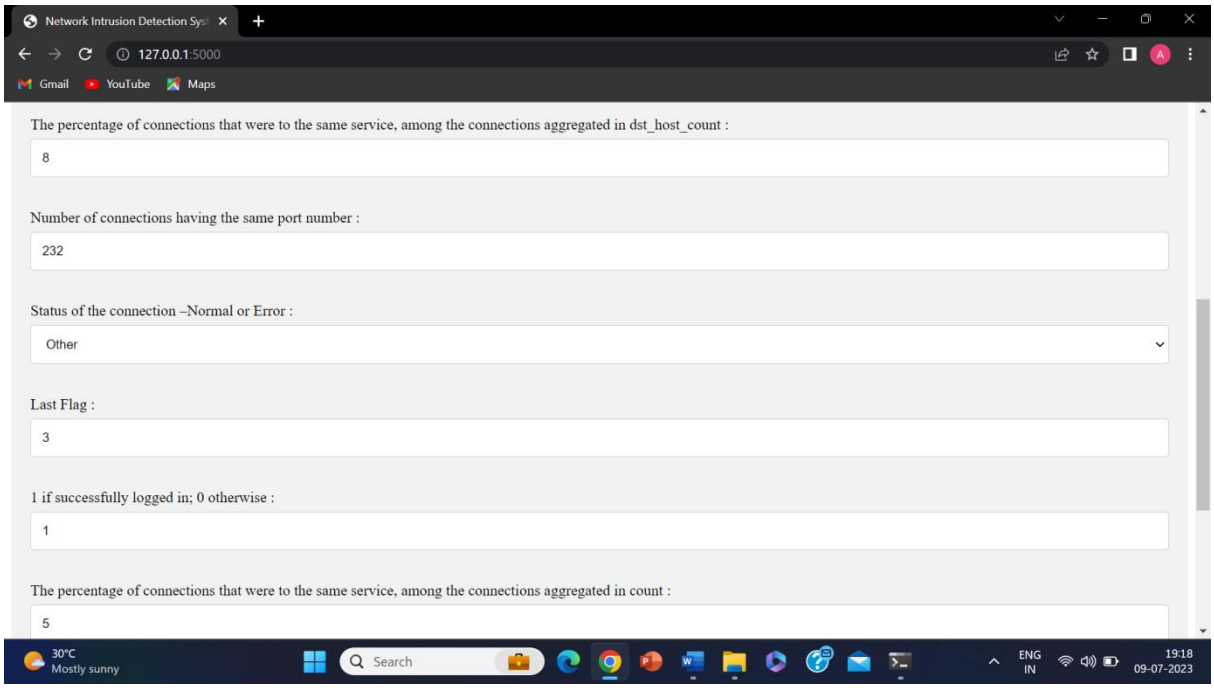

Network Intrusion Detection Syst
127.0.0.1:5000
Gmail
YouTube
Maps
Network Intrusion Detection System
Attack:
Other
Number of connections to the same destination host as the current connection in the past two seconds :
16
The percentage of connections that were to different services, among the connections aggregated in dst_host_count :
6
The percentage of connections that were to the same source port, among the connections aggregated in dst_host_srv_count :
324
The percentage of connections that were to the same service, among the connections aggregated in dst_host_count :
0
31°C
Partly sunny
Search
ENG
IN
18:50
09-07-2023

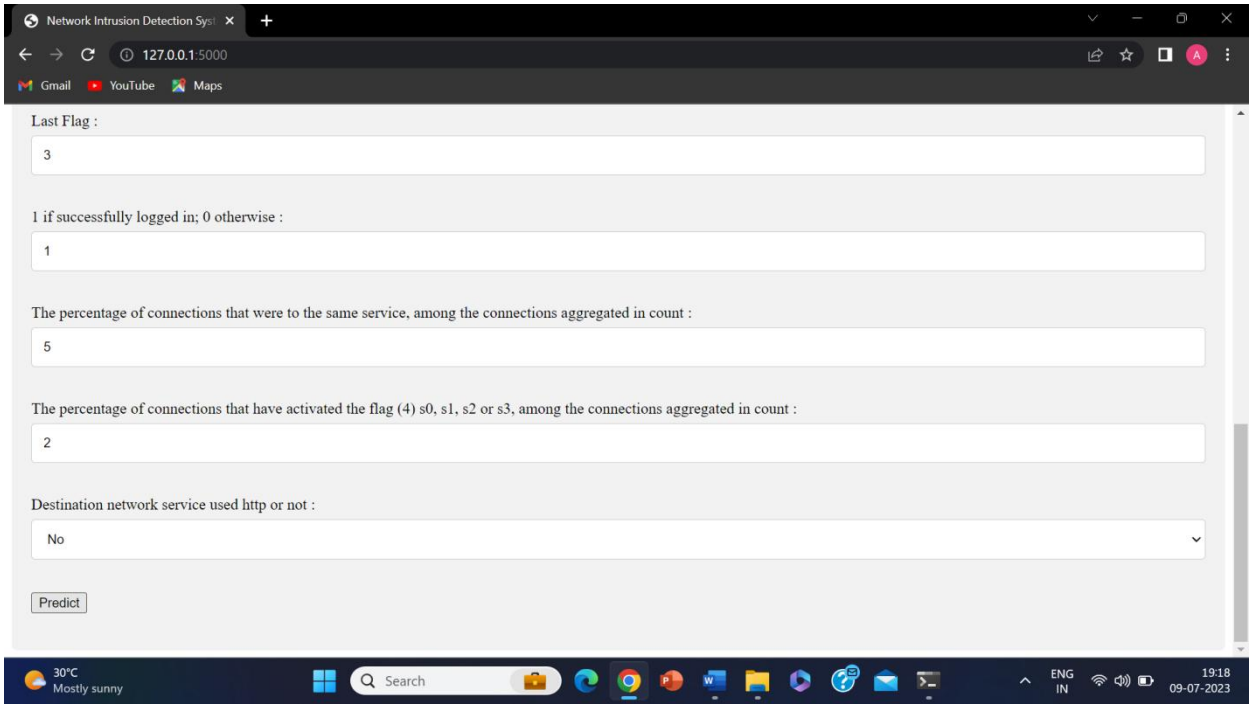

Network Intrusion Detection Syst
127.0.0.1:5000
Gmail
YouTube
Maps
Number of connections having the same port number :
0
Status of the connection –Normal or Error :
SF
Last Flag :
22
1 if successfully logged in; 0 otherwise :
0
The percentage of connections that were to the same service, among the connections aggregated in count :
0
The percentage of connections that have activated the flag (4) s0, s1, s2 or s3, among the connections aggregated in count :
0
31°C
Partly sunny
Search
ENG
IN
18:50
09-07-2023

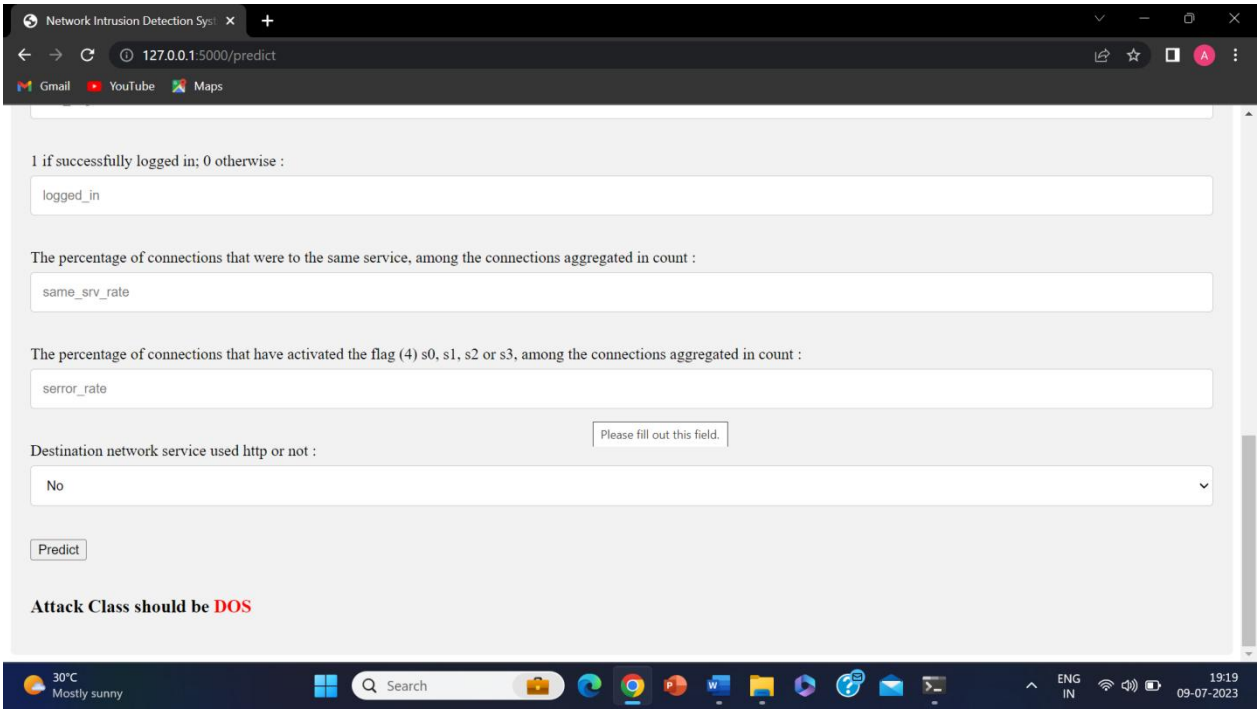

Network Intrusion Detection Syst
127.0.0.1:5000
Gmail
YouTube
Maps
Last Flag :
22
1 if successfully logged in; 0 otherwise :
0
The percentage of connections that were to the same service, among the connections aggregated in count :
0
The percentage of connections that have activated the flag (4) s0, s1, s2 or s3, among the connections aggregated in count :
0
Destination network service used http or not :
No
Predict
31°C
Partly sunny
Search
ENG
IN
18:50
09-07-2023


Network Intrusion Detection Syst
127.0.0.1:5000/predict
Gmail
YouTube
Maps
1 if successfully logged in; 0 otherwise :
logged_in
The percentage of connections that were to the same service, among the connections aggregated in count :
same_srv_rate
The percentage of connections that have activated the flag (4) s0, s1, s2 or s3, among the connections aggregated in count :
serror_rate
Destination network service used http or not :
No
Predict
Attack Class should be PROBE
31°C
Partly sunny
Search
ENG
IN
18:51
09-07-2023


Network Intrusion Detection Syst
127.0.0.1:5000
Gmail
YouTube
Maps
Network Intrusion Detection System
Attack:
neptune
Number of connections to the same destination host as the current connection in the past two seconds :
23
The percentage of connections that were to different services, among the connections aggregated in dst_host_count :
2
The percentage of connections that were to the same source port, among the connections aggregated in dst_host_srv_count :
5
The percentage of connections that were to the same service, among the connections aggregated in dst_host_count :
8
30°C
Mostly sunny
Search
ENG
IN
19:18
09-07-2023


Network Intrusion Detection Syst
127.0.0.1:5000
Gmail
YouTube
Maps
The percentage of connections that were to the same service, among the connections aggregated in dst_host_count :
8
Number of connections having the same port number :
232
Status of the connection –Normal or Error :
Other
Last Flag :
3
1 if successfully logged in; 0 otherwise :
1
The percentage of connections that were to the same service, among the connections aggregated in count :
5
30°C
Mostly sunny
Search
ENG
IN
19:18
09-07-2023


Network Intrusion Detection Syst
127.0.0.1:5000
Gmail
YouTube
Maps
Last Flag :
3
1 if successfully logged in; 0 otherwise :
1
The percentage of connections that were to the same service, among the connections aggregated in count :
5
The percentage of connections that have activated the flag (4) s0, s1, s2 or s3, among the connections aggregated in count :
2
Destination network service used http or not :
No
Predict
30°C
Mostly sunny
Search
ENG
IN
19:18
09-07-2023


Network Intrusion Detection Syst
127.0.0.1:5000/predict
Gmail
YouTube
Maps
1 if successfully logged in; 0 otherwise :
logged_in
The percentage of connections that were to the same service, among the connections aggregated in count :
same_srv_rate
The percentage of connections that have activated the flag (4) s0, s1, s2 or s3, among the connections aggregated in count :
serror_rate
Please fill out this field.
Destination network service used http or not :
No
Predict
Attack Class should be DOS
30°C
Mostly sunny
Search
ENG
IN
19:19
09-07-2023

CHAPTER 8
SYSTEM TESTING AND IMPLEMENTATION

8.1 Introduction

It is done by the developer itself in every stage of the project and fine-tuning the bug and module predicated additionally done by the developer only here we are going to solve all the runtime errors

MANUAL TESTING

As our Project is academic Leave, we can do any automatic testing so we follow manual testing by endeavor and error methods.

Testing

Testingisaprocessofexecutingaprogramwiththeaimoffindingerror.Tomakeoursoftware perform well it should be error free. If testing is done successfully, it will remove all the errors from the software.

Types of Testing

1. White Box Testing
2. Black Box Testing
3. Unit testing
4. Integration Testing
5. Alpha Testing
6. Beta Testing
7. Performance Testing and so on

White Box Testing

Testing technique based on knowledge of the internal logic of an application's code and includes tests like coverage of code statements, branches, paths, conditions. It is performed by software developers

Black Box Testing

A method of software testing that verifies the functionality of an application without having specific knowledge of the application’s code / internal structure. Tests are

based on requirements and functionality.

Unit Testing

Software verification and validation method in which a programmer tests if individual units of source code are fit for use. It is usually conducted by the development team.

Integration Testing

The phase in software testing in which individual software modules are combined and tested as a group. It is usually conducted by testing teams.

Alpha Testing

Type of testing a software product or system conducted at the developer's site. Usually it is performed by the end users.

BetaTesting

Final testing before releasing application for commercial purpose. It is typically done by end- users or others.

PerformanceTesting

Functional testing conducted to evaluate the compliance of a system or component with specified performance requirements. It is usually conducted by the performance engineer.

Black Box Testing

Blackbox testing is testing the functionality of an application without knowing the details of its implementation including internal program structure, data structure sets Test cases for black box testing are created based on the requirement specifications. Therefore, it is also called as specification-based testing. Fig.4.1 represents the black box testing:

Fig.: Black Box Testing

When applied to machine learning models, black box testing would mean testing machine learning models without knowing the internal details such as features of the machine learning

model, the algorithm used to create the model etc. The challenge, however, is to verify the test outcome against the expected values that are known beforehand.

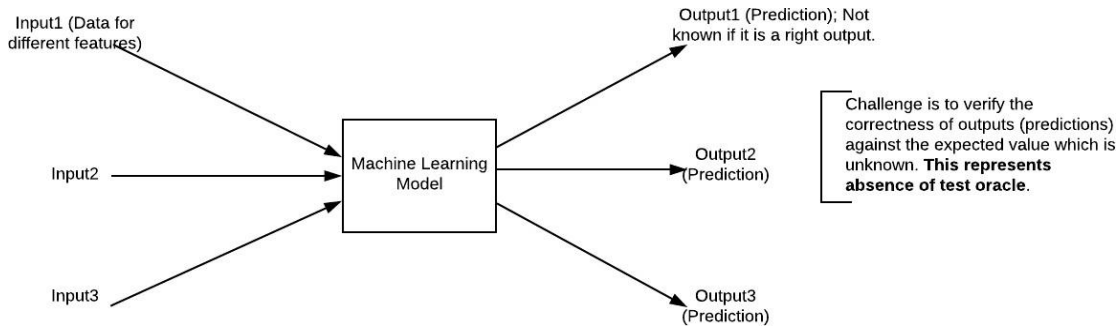


Fig.: Black Box Testing for Machine Learning algorithms

The above Fig.4.2 represents the black box testing procedure for machine learning algorithms.

Table.4.1: Black box Testing

Input	**Actual Output**	**Predicted Output**
[16,6,324,0,0,0,22,0,0,0,0,0,0]	0	0
[16,7,263,7,0,2,700,9,10,1153,832,9,2]	1	1

The model gives out the correct output when different inputs are given which are mentioned in Table 4.1. Therefore, the program is said to be executed as expected or correct program

TESTING UNITTESTING

It is done by the developer itself in every stage of the project and fine-tuning the bug and module predicated additionally done by the developer only here we are going to solve all the runtime errors.

MANUAL TESTING

As our Project is academic Leave, we can do any automatic testing so we follow manual testing by endeavor and error methods.

DEPLOYMENT OF SYSTEM AND MAINTENANCE

Once the project is total yare, we will come to deployment of client system in genuinely world as its academic leave we did deployment i our college lab only with all need Software's with having Windows OS.

The Maintenance of our Project is one-time process only

IMPLEMENTATION

The Implementation is Phase where we endeavor to give the practical output of the work done in designing stage and most of Coding in Business logic lay coms into action in this stage its main and crucial part of the project.

CHAPTER 9
CONCLUSION

Right now, estimations of help vector machine, ANN, CNN, Random Forest and profound learning calculations dependent on modern CICIDS2017 dataset were introduced relatively. Results show that the profound learning calculation performed fundamentally preferable outcomes over SVM, ANN, RF and CNN. We are going to utilize port sweep endeavors as well as other assault types with AI and profound learning calculations, Apache Hadoop, and sparkle innovations together dependent on this dataset later on. All these calculation helps us to detect the cyber attack in network. It happens in the way that when we consider long back years there may be so many attacks happened so when these attacks are recognized then the features at which values these attacks are happening will be stored in some datasets. So by using these datasets we are going to predict whether cyber attack is done or not. These predictions can be done by four algorithms like SVM, ANN, RF, CNN this paper helps to identify which algorithm predicts the best accuracy rates which helps to predict best results to identify the cyber attacks happened or not.

CHAPTER 10
FUTURE ENHANCEMENT

In enhancement we will add some ML Algorithms to increase accuracy.
The algorithms like **SGD algorithm** ML algorithm can be used as the future enhancement for this project

- Improved Training Efficiency: The SGD algorithm enables faster convergence during the training process compared to traditional gradient descent methods. This is especially beneficial when dealing with large-scale datasets, as it reduces the computational burden and speeds up the model training.
- Handling Large Datasets: With the SGD algorithm, models can efficiently handle datasets that do not fit into memory. By randomly selecting mini-batches, the algorithm can iteratively update the model's parameters, making it feasible to train on data that would otherwise be too large to process.
- Robustness to Noisy Data: The stochastic nature of SGD allows the model to adapt and handle noisy or non-linear data more effectively. It can help prevent overfitting by introducing some randomness into the learning process, leading to better generalization and improved performance on unseen data.
- Scalability: The SGD algorithm scales well with increasing data volumes, making it suitable for real-time or large-scale cyber attack detection scenarios. It can efficiently handle continuous streams of data and update the model's parameters incrementally, allowing for adaptive and responsive detection capabilities.

CHAPTER 11

REFERENCES

[1] K. Graves, Ceh: Official certified ethical hacker review guide: Exam 312-50. John Wiley & Sons, 2007.

[2] R. Christopher, “Port scanning techniques and the defense against them,” SANS Institute, 2001.

[3] M. Baykara, R. Das¸, and I. Karado ˘gan, “Bilgi g ¨uvenli ˘gi sistemlerinde kullanilan arac¸larin incelenmesi,” in 1st International Symposium on Digital Forensics and Security (ISDFS13), 2013, pp. 231–239.

[4] S. Staniford, J. A. Hoagland, and J. M. McAlerney, “Practical automated detection of stealthy portscans,” Journal of Computer Security, vol. 10, no. 1-2, pp. 105–136, 2002.

[5] S. Robertson, E. V. Siegel, M. Miller, and S. J. Stolfo, “Surveillance detection in high bandwidth environments,” in DARPA Information Survivability Conference and Exposition, 2003. Proceedings, vol. 1. IEEE, 2003, pp. 130–138.

[6] K. Ibrahimi and M. Ouaddane, “Management of intrusion detection systems based-kdd99: Analysis with lda and pca,” in Wireless Networks and Mobile Communications (WINCOM), 2017 International Conference on. IEEE, 2017, pp. 1–6.

[7] N. Moustafa and J. Slay, “The significant features of the unsw-nb15 and the kdd99 data sets for network intrusion detection systems,” in Building Analysis Datasets and Gathering

Experience Returns for Security (BADGERS), 2015 4th International Workshop on. IEEE, 2015, pp. 25–31.

[8] L. Sun, T. Anthony, H. Z. Xia, J. Chen, X. Huang, and Y. Zhang, “Detection and classification of malicious patterns in network traffic using benford’s law,” in Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), 2017. IEEE, 2017, pp. 864–872.

[9] S. M. Almansob and S. S. Lomte, “Addressing challenges for intrusion detection system using naive bayes and pca algorithm,” in Convergence in Technology (I2CT), 2017 2nd International Conference for. IEEE, 2017, pp. 565–568.

[10] M. C. Raja and M. M. A. Rabbani, “Combined analysis of support vector machine and principle component analysis for ids,” in IEEE International Conference on Communication and Electronics Systems, 2016, pp. 1–5.

[11] S. Aljawarneh, M. Aldwairi, and M. B. Yassein, “Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model,” Journal of Computational Science, vol. 25, pp. 152–160, 2018.

[12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization.” in ICISSP, 2018, pp. 108–116.

[13] D. Aksu, S. Ustebay, M. A. Aydin, and T. Atmaca, “Intrusion detection with comparative analysis of supervised learning techniques and fisher score feature selection algorithm,” in International Symposium on Computer and Information Sciences. Springer, 2018, pp. 141–149.

[14] N. Marir, H. Wang, G. Feng, B. Li, and M. Jia, “Distributed abnormal behavior detection approach based on deep belief network and ensemble svm using spark,” IEEE Access, 2018.

[15] P. A. A. Resende and A. C. Drummond, “Adaptive anomaly-based intrusion detection system using genetic algorithm and profiling,” Security and Privacy, vol. 1, no. 4, p. e36, 2018.

[16] C. Cortes and V. Vapnik, “Support-vector networks,” Machine learning, vol. 20, no. 3, pp. 273–297, 1995.

[17] R. Shouval, O. Bondi, H. Mishan, A. Shimoni, R. Unger, and A. Nagler, “Application of machine learning algorithms for clinical predictive modeling: a data-mining approach in sct,” Bone marrow transplantation, vol. 49, no. 3, p. 332, 2014